CHAPTER XVI11

Artificial Intelligence: Retrospects

Turing

IN 1950, ALAN TURING wrote a most prophetic and provocative article on Artificial Intelligence. It was entitled "Computing Machinery and Intelligence" and appeared in the journal *Mind*.' I will say some things about that article, but I would like to precede them with some remarks about Turing the man.

Alan Mathison Turing was born in London in 1912. He was a child full of curiosity and humor. Gifted in mathematics, he went to Cambridge where his interests in machinery and mathematical logic cross-fertilized and resulted in his famous paper on "computable numbers", in which he invented the theory of Turing machines and demonstrated the unsolvability of the halting problem; it was published in 1937. In the 1940's, his interests turned from the theory of computing machines to the actual building of real computers. He was a major figure in the development of computers in Britain, and a staunch defender of Artificial In-

FIGURE 113. *Alan Turing, after a successful race (May, 1950). [From Sara Turing, Alan M. Turing (Cambridge, U. K.:W. Heffer & Sons, 1959).*

telligence when it first came under attack. One of his best friends was David Champernowne (who later worked on computer composition of music). Champernowne and Turing were both avid chess players and invented "round-the-house" chess: after your move, run around the house-if you get back before your opponent has moved, you're entitled to another move. More seriously, Turing and Champernowne invented the first chess playing program, called "Turochamp . Turing died young, at 41-apparently of an accident with chemicals. Or some say suicide. His mother, Sara Turing, wrote his biography. From the people she quotes, one gets the sense that Turing was highly unconventional, even gauche in some ways, but so honest and decent that he was vulnerable to the world. He loved games, chess, children, and bike riding; he was a strong long-distance runner. As a student at Cambridge, he bought himself a second-hand violin and taught himself to play. Though not very musical, he derived a great deal of enjoyment from it. He was somewhat eccentric, given to great bursts of energy in the oddest directions. One area he explored was the problem of morphogenesis in biology. According to his mother, Turing "had a particular fondness for the *Pickwick Papers*", but "poetry, with the exception of Shakespeare's, meant nothing to him." Alan Turing was one of the true pioneers in the field of computer science.

The Turing Test

Turing's article begins with the sentence: "I propose to consider the question `Can machines think?'" Since, as he points out, these are loaded terms, it is obvious that we should search for an operational way to approach the question. This, he suggests, is contained in what he calls the "imitation game"; it is nowadays known as the *Turing test*. Turing introduces it as follows:

> It is played with three people: a man (A), a woman (B), and an interrogator (C) who may be of either sex. The interrogator stays in a room apart from the other two. The object of the game for the interrogator is to determine which of the other two is the man and which is the woman. He knows them by labels X and Y, and at the end of the game he says either "X is A and Y is B" or "X is B and Y is A". The interrogator is allowed to put questions to A and B thus:
>
> C: Will X please tell me the length of his or her hair?
>
> Now suppose X is actually A, then A must answer. It is A's object in the game to try to cause C to make the wrong identification. His answer might therefore be
>
> "My hair is shingled, and the longest strands are about nine inches long."
>
> In order that tones of voice may not help the interrogator the answers should be written, or better still, typewritten. The ideal arrangement is to have a teleprinter communicating between the two rooms. Alternatively the questions and answers can be repeated by an intermediary. The object of the game for the third player (B) is to help the interrogator. The best strategy for her is probably to give truthful answers. She can add such things as "I am the woman, don't listen to him!" to her answers, but it will avail nothing as the man can make similar remarks.

> We now ask the question, "What will happen when a machine takes the part of A in this game Will the interrogator decide wrongly as often when the game is played like this as he does when the game is played between a man and a woman? These questions replace our original, "Can machines think?"[2]

After having spelled out the nature of his test, Turing goes on to make some commentaries on it, which, given the year he was writing in, are quite sophisticated. To begin with, he gives a short hypothetical dialogue between interrogator and interrogatee:[3]

> Q. Please write me a sonnet on the subject of the Forth Bridge [a bridge over the Firth of Forth, in Scotland].
> A. Count me out on this one. I never could write poetry.
> Q. Add 34957 to 70764.
> A. (Pause about 30 seconds and then give as answer) 105621.
> Q. Do you play chess?
> A. Yes.
> Q. I have K at my K1, and no other pieces. You have only K at K6 and R at R1. It is your move. What do you play?
> A. (After a pause of 15 seconds) R-R8 mate.

Few readers notice that in the arithmetic problem, not only is there an inordinately long delay, but moreover, the answer given is wrong! This would be easy to account for if the respondent were a human: a mere calculational error. But if the respondent were a machine, a variety of explanations are possible. Here are some:

> (1) a run-time error on the hardware level (i.e., an irreproducible fluke);
> (2) an unintentional hardware (or programming) (reproducibly) causes arithmetical mistakes;
> (3) a ploy deliberately inserted by the machine's programmer (or builder) to introduce occasional arithmetical mistakes, so as to trick interrogators;
> (4) an unanticipated epiphenomenon: the program has a hard time thinking abstractly, and simply made "an honest mistake", which it might not make the next time around;
> (5) a joke on the part of the machine itself, deliberately teasing its interrogator.

Reflection on what Turing might have meant by this subtle touch opens up just about all the major philosophical issues connected with Artificial Intelligence.

Turing goes on to point out that

> The new problem has the advantage of drawing a fairly sharp line between the physical and the intellectual capacities of a man. . . . We do not wish to penalize the machine for its inability to shine in beauty competitions, nor to penalize a man for losing in a race against an airplane.'

One of the pleasures of the article is to see how far Turing traced out each

line of thought, usually turning up a seeming contradiction at some stage and, by refining his concepts, resolving it at a deeper level of analysis. Because of this depth of penetration into the issues. the article still shines after nearly thirty years of tremendous progress in computer development and intensive work in Al. In the following short excerpt you can see some of this rich back-and-forth working of ideas:

> The game may perhaps be criticized on the ground that the odds are weighted too heavily against the machine. If the man were to try to pretend to be the machine he would clearly make a very poor showing. He would be given away at once by slowness and inaccuracy in arithmetic. May not machines carry out something which ought to be described as thinking but which is very different from what a man does: This objection is a very strong one, but at least we can say that if, nevertheless, a machine can be constructed to play the imitation game satisfactorily, we need not be troubled by this objection.
>
> It might be urged that when playing the "imitation game" the best strategy for the machine may possibly be something other than imitation of the behaviour of a man. This may be, but I think it is unlikely that there is any greet effect of this kind. In any case there is ,no intention to investigate here the theory of the game, and it will be assumed that the best strategy is to try to provide answers that would naturally be given by a mans

Once the test has been proposed and discussed, Turing remarks:

> The original question "Can machines think 1 believe to be too meaningless to deserve discussion. Nevertheless, I believe that at the end of the century the use of words and general educated opinion will have altered so much that one will be able to speak of machines thinking without expecting to be contradicted.[6]

Turing Anticipates Objections

Aware of the storm of opposition that would undoubtedly greet this opinion, he then proceeds to pick apart, concisely and with wry humor, a series of objections to the notion that machines could think. Below I list the nine types of objections he counters, using his own descriptions of them .7 Unfortunately there is not space to reproduce the humorous and ingenious responses he formulated. You may enjoy pondering the objections yourself, and figuring out your own responses.

(1) The Theological Objection. Thinking is a function of man's immortal soul. God has given an immortal soul to every man and woman, but not to any other animal or to machines. Hence no animal or machine can think.
(2) The "Heads in the Sand" Objection. The consequences of machines thinking would be too dreadful. Let us hope and believe that they cannot do so.
(3) The Mathematical Objection. [This is essentially the Lucas argument.
(4) The Argument from Consciousness. "Not until a machine can write a sonnet or compose a concerto because of thoughts and emotions felt, and not by the chance fall of symbols, could we agree that machine equals brainy that is, not only write it but know that it had written it. No mechanism

could feel (and not merely artificially signal, an easy contrivance) pleasure at its successes, grief when its valves fuse, be warmed by flattery, be made miserable by its mistakes, be charmed by sex, be angry or depressed when it cannot get what it wants." [A quote from a certain Professor Jefferson.]

Turing is quite concerned that he should answer this serious objection in full detail. Accordingly, he devotes quite a bit of space to his answer, and in it he offers another short hypothetical dialogue:'

Interrogator: In the first line of your sonnet which reads "Shall I compare thee to a summer's day", would not "a spring day" do as well or better, Witness: It wouldn't scan.

Interrogator: How about ' a winter's day"? That would scan all right. Witness: Yes, but nobody wants to be compared to a winter's day. Interrogator: Would you say Mr. Pickwick reminded you of Christmas? Witness: In a way.

Interrogator: Yet Christmas is a winter's day, and I do not think Mr. Pickwick would mind the comparison.

Witness: I don't think you're serious. By a winter's day one means a typical winter's day, rather than a special one like Christmas.

After this dialogue, Turing asks, "What would Professor Jefferson say if the sonnet-writing machine was able to answer like this in the *viva voce*?"

Further objections:

(5) Arguments from various Disabilities. These arguments take the form, "I grant you that you can make machines do all the things that you have mentioned but you will never be able to make one to do X." Numerous features X are suggested in this connection. I offer a selection:

Be kind, resourceful, beautiful, friendly, have initiative, have a sense of humor, tell right from wrong, make mistakes, fall in love, enjoy strawberries and cream, make someone fall in love with it, learn from experience, use words properly, be the subject of its own thought, have as much diversity of behaviour as a man, do something really new.

(6) Lady Lovelace's Objection. Our most detailed information of Babbage's Analytical Engine comes from a memoir by Lady Lovelace. In it she states, "The Analytical Engine has no pretensions to originate anything. It can do whatever we know how to order it to perform" (her italics).

(7) Argument from Continuity in the Nervous System. The nervous system is certainly not a discrete state machine. A small error in the information about the size of a nervous impulse impinging on a neuron may make a large difference to the size of the outgoing impulse. It may be argued that, this being so, one cannot expect to be able to mimic the behaviour of the nervous system with a discrete state system.

(8) The Argument from Informality of Behaviour. It seems to run something like this. "If each man had a definite set of rules of conduct by which he regulated his life he would be no better than a machine. But there are no such rules, so men cannot be machines."

(9) The Argument from Extra-Sensory Perception. Let us play the imitation game, using as witnesses a man who is good as a telepathic receiver, and a digital computer. The interrogator can ask such questions as "What suit

does the card in my right hand belong to?" The man by telepathy or clairvoyance gives the right answer 130 times out of 400 cards. The machine can only guess at random, and perhaps get 104 right, so the interrogator makes the right identification.

As you can see, many of the objections overlap and are related to each other. I have attempted in this book to answer most of them in some way or other, usually far less concisely than Turing did.

Objection (9) I find remarkable. I have seen the Turing paper reprinted in a book-but with objection (9) omitted-which I find equally remarkable. My own point of view-contrary to Turing's-is that ESP does not exist. Turing was reluctant to accept the idea that ESP is real, but did so nonetheless, being compelled by his outstanding scientific integrity to accept the consequences of what he viewed as powerful statistical evidence in favor of ESP. I disagree, though I consider it an exceedingly complex and fascinating question.

"Parry Encounters the Doctor"

In the Dialogue preceding this Chapter, you have seen an authentic exchange between a computer program and a human. Now there have been many programs with less ability to "converse" than SHRDLU. Two rather famous ones are "Doctor", created by Joseph Weizenbaum, and "Parry", created by Kenneth Colby. The former is supposed to simulate a psychiatrist using "nondirective therapy", the latter to simulate the belief structure of a paranoid. An amusing head-on collision between the two programs was arranged over a computer network, and the following is a short excerpt from the full transcript:[9]

Parry: I don't understand your motives.
Doctor: Why don't you understand my motives?
Parry: Shouldn't I be?
Doctor: Why do you say that just now?
Parry: That's just the way it is.
Doctor: Suppose that's just the way it were not.
Parry: So?
Doctor: I am not sure I understand you fully.
Parry: You harp on the same questions too much.
Doctor: What does that similarity suggest to you?
Parry: I don't get you.
Doctor: Do you wish that you did get I?
Parry: It's about time you showed some interest in my feelings.
Doctor: Oh? I showed some interest in your feelings?

These programs, of course, were made to talk to humans, so they are shown up rather poorly here, pitted against each other. Mostly they rely on shrewd guesses as to the nature of the input (which they analyze quite shallowly) and spit back canned answers that have been carefully selected from a large repertoire. The answer may be only partially canned: for example, a template with blanks that can be filled in. It is assumed that their

human partners will read much more into what they say than is actually underlying it. And in fact, according to Weizenbaum, in his book Computer Power and Human Reason, just that happens. He writes:

> ELIZA [the program from which Doctor was made created the most remarkable illusion of having understood in the minds of the many people who conversed with it.... They would often demand to be permitted to converse with the system in private, and would, after conversing with it for a time, insist, in spite of my explanations, that the machine really understood them.[10]

Given the above excerpt, you may find this incredible. Incredible, but true. Weizenbaum has an explanation:

> Most men don't understand computers to even the slightest degree. So, unless they are capable of very great skepticism (the kind we bring to bear while watching a stage magician), they can explain the computer's intellectual feats only by bringing to hear the single analogy available to them, that is, their model of their own capacity to think. No wonder, then, that they overshoot the mark: it is truly impossible to imagine a human who could imitate ELIZA, for example, but for whom ELIZA's language abilities were his limit."

Which amounts to an admission that this kind of program is based on a shrewd mixture of bravado and bluffing, taking advantage of people's gullibility.

In light of this weird "ELIZA-effect", some people have suggested that the Turing test needs revision, since people can apparently be fooled by simplistic gimmickry. It has been suggested that the interrogator should be a Nobel Prize-winning scientist. It might be more advisable to turn the Turing test on its head, and insist that the interrogator should be another computer. Or perhaps there should be two interrogators-a human and a computer-and one witness, and the two interrogators should try to figure out whether the witness is a human or a computer.

In a more serious vein, I personally feel that the Turing test, as originally proposed, is quite reasonable. As for the people who Weizenbaum claims were sucked in by ELIZA, they were not urged to be skeptical, or to use all their wits in trying to determine if the "person" typing to them were human or not. I think that Turing's insight into this issue was sound, and that the Turing test, essentially unmodified, will survive.

A Brief History of AI

I would like in the next few pages to present the story, perhaps from an unorthodox point of view, of some of the efforts at unraveling the algorithms behind intelligence: there have been failures and setbacks and there will continue to be. Nonetheless, we are learning a great deal, and it is an exciting period.

Ever since Pascal and Leibniz, people have dreamt of machines that could perform intellectual tasks. In the nineteenth century, Boole and De Morgan devised "laws of thought"-essentially the Propositional

Calculus-and thus took the first step towards At software; also Charles Babbage designed the first "calculating engine"-the precursor to the hardware of computers and hence of AI. One could define AI as coming into existence at the moment when mechanical devices took over any tasks previously performable only by human minds. It is hard to look back and imagine the feelings of those who first saw toothed wheels performing additions and multiplications of large numbers. Perhaps they experienced a sense of awe at seeing "thoughts" flow in their very physical hardware. In any case, we do know that nearly a century later, when the first electronic computers were constructed, their inventors did experience an awesome and mystical sense of being in the presence of another kind of "thinking being". To what extent real thought was taking place was a source of much puzzlement; and even now, several decades later, the question remains a great source of stimulation and vitriolics.

It is interesting that nowadays, practically no one feels that sense of awe any longer-even when computers perform operations that are incredibly more sophisticated than those which sent thrills down spines in the early days. The once-exciting phrase "Giant Electronic Brain" remains only as a sort of "camp" cliché, a ridiculous vestige of the era of Flash Gordon and Buck Rogers. It is a bit sad that we become blasé so quickly.

There is a related "Theorem" about progress in Al: once some mental function is programmed, people soon cease to consider it as an essential ingredient of "real thinking". The ineluctable core of intelligence is always in that next thing which hasn't yet been programmed. This "Theorem" was first proposed to me by Larry Tesler, so I call it Tesler's Theorem. "Al is whatever hasn't been done vet."

A selective overview of AI is furnished below. It shows several domains in which workers have concentrated their efforts, each one seeming in its own way to require the quintessence of intelligence. With some of the domains I have included a breakdown according to methods employed, or more specific areas of concentration.

- mechanical translation
  - direct (dictionary look-up with some word rearrangement)
  - indirect (via some intermediary internal language)
- game playing
  - chess
    - with brute force look-ahead
    - with heuristically pruned look-ahead
    - with no look-ahead checkers
  - go
  - kalah
  - bridge (bidding; playing)
  - poker
  - variations on tic-tac-toe
  - etc.

proving theorems in various parts. of mathematics
- symbolic logic
  - "resolution" theorem-proving
  - elementary geometry

symbolic manipulation of mathematical expressions
- symbolic integration
- algebraic simplification
- summation of infinite series

vision
- printed matter:
  - recognition of individual hand-printed characters drawn from a small class (e.g., numerals)
  - reading text in variable fonts reading passages in handwriting
  - reading Chinese or Japanese printed characters
  - reading Chinese or Japanese handwritten characters
- pictorial:
  - locating prespecified objects in photographs
  - decomposition of a scene into separate objects
  - identification of separate objects in a scene
  - recognition of objects portrayed in sketches by people
  - recognition of human faces

hearing
- understanding spoken words drawn from a limited vocabulary (e.g., names of the ten digits)
- understanding continuous speech in fixed domains finding boundaries between phonemes
- identifying phonemes
- finding boundaries between morphemes
- identifying morphemes
- putting together whole words and sentences

understanding natural languages
- answering questions in specific domains
- parsing complex sentences
- making paraphrases of longer pieces of text
- using knowledge of the real world in order to understand passages
- resolving ambiguous references

producing natural language
- abstract poetry (e.g., haiku)
- random sentences, paragraphs, or longer pieces of text producing output from internal representation of knowledge

creating original thoughts or works of *art*
- poetry writing (haiku) story writing
- computer art
- musical composition
  - atonal
  - tonal

analogical thinking
- geometrical shapes ("intelligence tests")
- constructing proofs in one domain of mathematics based on those in a related domain

learning
- adjustment of parameters
- concept formation

Mechanical Translation

Many of the preceding topics will not be touched upon in my selective discussion below, but the list would not be accurate without them. The first few topics are listed in historical order. In each of them, early efforts fell short of expectations. For example, the pitfalls in mechanical translation came as a great surprise to many who had thought it was a nearly straightforward task, whose perfection, to be sure, would be arduous, but whose basic implementation should be easy. As it turns out, translation is far more complex than mere dictionary look-up and word rearranging. Nor is the difficulty caused by a lack of knowledge of idiomatic phrases. The fact is that translation involves having a mental model of the world being discussed, and manipulating symbols in that model. A program which makes no use of a model of the world as it reads the passage will soon get hopelessly bogged down in ambiguities and multiple meanings. Even people-who have a huge advantage over computers, for they come fully equipped with an understanding of the world-when given a piece of text and a dictionary of a language they do not know, find it next to impossible to translate the text into their own language. Thus-and it is not surprising in retrospect-the first problem of AI led immediately to the issues at the heart of AI.

Computer Chess

Computer chess, too, proved to be much more difficult than the early intuitive estimates had suggested. Here again it turns out that the way humans represent a chess situation in their minds is far more complex than just knowing which piece is on which square, coupled with knowledge of the rules of chess. It involves perceiving configurations of several related pieces, as well as knowledge of heuristics, or rules of thumb, which pertain to

such higher-level chunks. Even though heuristic rules are not rigorous in the way that the official rules are, they provide shortcut insights into what is going on on the board, which knowledge of the official rules does not. This much was recognized from the start; it was simply underestimated how large a role the intuitive, chunked understanding of the chess world plays in human chess skill. It was predicted that a program having some basic heuristics, coupled with the blinding speed and accuracy of a computer to look ahead in the game and analyze each possible move, would easily beat top-flight human players-a prediction which, even after twenty-five years of intense work by various people, still is far from being realized.

People are nowadays tackling the chess problem from various angles. One of the most novel involves the hypothesis that looking ahead is a silly thing to do. One should instead merely look at what is on the board at present, and, using some heuristics, generate a plan, and then find a move which advances that particular plan. Of course, rules for the formulation of chess plans will necessarily involve heuristics which are, in some sense, "flattened" versions of looking ahead. That is, the equivalent of many games' experience of looking ahead is "squeezed" into another form which ostensibly doesn't involve looking ahead. In some sense this is a game of words. But if the "flattened" knowledge gives answers more efficiently than the actual look-ahead-even if it occasionally misleads- then something has been gained. Now this kind of distillation of knowledge into more highly usable forms is just what intelligence excels at-so look-ahead-less chess is probably a fruitful line of research to push. Particularly intriguing would be to devise a program which itself could convert knowledge gained from looking ahead into "flattened" rules-but that is an immense task.

Samuel's Checker Program

As a matter of fact, such a method was developed by Arthur Samuel in his admirable checker-playing program. Samuel's trick was to use both dynamic (look-ahead) and static (no-look-ahead) ways of evaluating any given board position. The static method involved a simple mathematical function of several quantities characterizing any board position, and thus could be calculated practically instantaneously, whereas the dynamic evaluation method involved creating a "tree" of possible future moves, responses to them, responses to the responses, and so forth (as was shown in Fig. 38). In the static evaluation function there were some parameters which could vary; the effect of varying them was to provide a set of different possible versions of the static evaluation function. Samuel's strategy was to select, in an evolutionary way, better and better values of those parameters.

Here's how this was done: each time the program evaluated a board position, it did so both statically and dynamically. The answer gotten by looking ahead-let us call it **D**-was used in determining the move to be made. The purpose of S, the static evaluation, was trickier: on each move, the variable parameters were readjusted slightly so that S approximated **D**

as accurately as possible. The effect was to partially encode in the values of the static evaluation's parameters the knowledge gained by dynamically searching the tree. In short, the idea was to "flatten" the complex dynamic evaluation method into the much simpler and more efficient static evaluation function.

There is a rather nice recursive effect here. The point is that the dynamic evaluation of any single board position involves looking ahead a finite number of moves-say seven. Now each of the scads of board positions which might turn up seven turns down the road has to be itself evaluated somehow as well. But when the program evaluates these positions, it certainly cannot look another seven moves ahead, lest it have to look fourteen positions ahead, then twenty-one, etc., etc.-an infinite regress. Instead, it relies on static evaluations of positions seven moves ahead. Therefore, in Samuel's scheme, an intricate sort of feedback takes place, wherein the program is constantly trying to "flatten" look-ahead evaluation into a simpler static recipe; and this recipe in turn plays a key role in the dynamic look-ahead evaluation. Thus the two are intimately linked together, and each benefits from improvements in the other in a recursive way.

The level of play of the Samuel checkers program is extremely high: of the order of the top human players in the world. If this is so, why not apply the same techniques to chess? An international committee, convened in 1961 to study the feasibility of computer chess, including the Dutch International Grandmaster and mathematician Max Euwe, came to the bleak conclusion that the Samuel technique would be approximately one million times as difficult to implement in chess as in checkers, and that seems to close the book on that.

The extraordinarily great skill of the checkers program cannot be taken as saying "intelligence has been achieved"; yet it should not be minimized, either. It is a combination of insights into what checkers is, how to think about checkers, and how to program. Some people might feel that all it shows is Samuel's own checkers ability. But this is not true, for at least two reasons. One is that skillful game players choose their moves according to mental processes which they do not fully understand-they use their intuitions. Now there is no known way that anyone can bring to light all of his own intuitions; the best one can do via introspection is to use "feeling" or "meta-intuition"-an intuition about one's intuitions-as a guide, and try to describe what one thinks one's intuitions are all about. But this will only give a rough approximation to the true complexity of intuitive methods. Hence it is virtually certain that Samuel has not mirrored his own personal methods of play in his program. The other reason that Samuel's program's play should not be confused with Samuel's own play is that Samuel does not play checkers as well as his program-it beats him. This is not a paradox at all-no more than is the fact that a computer which has been programmed to calculate 7T can outrace its programmer in spewing forth digits of π.

When Is a Program Original?

This issue of a program outdoing its programmer is connected with the question of "originality" in AI. What if an AI program comes up with an idea, or a line of play in a game, which its programmer has never entertained-who should get the credit? There are various interesting instances of this having happened, some on a fairly trivial level, some on a rather deep level. One of the more famous involved a program to find proofs of theorems in elementary Euclidean geometry, written by E. Gelernter. One day the program came up with a sparklingly ingenious proof of one of the basic theorems of geometry-the so-called "pons asinorum", or "bridge of asses".

This theorem states that the base angles of an isosceles triangle are equal. Its standard proof requires constructing an altitude which divides the triangle into symmetrical halves. The elegant method found by the program (see Fig. 114) used no construction lines. Instead, it considered

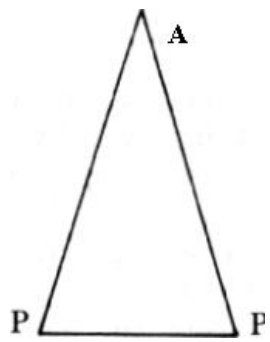


FIGURE 114. *Pons Asinorum Proof (found by Pappus [-300 A.D.] and Gelernter's program [--1960 A.D.]). Problem: To show that the base angles of an isosceles triangle are equal. Solution: As the triangle is isosceles, AP and AP' are of equal length. Therefore triangles PAP' and PAP are congruent (side-side-side). This implies that corresponding angles are equal. In particular, the two base angles are equal.*

the triangle and its mirror image as two different triangles. Then, having proved them congruent, it pointed out that the two base angles matched each other in this congruence-QED.

This gem of a proof delighted the program's creator and others; some saw evidence of genius in its performance. Not to take anything away from this feat, it happens that in A.D. 300 the geometer Pappus had actually found this proof, too. In any case, the question remains: "Who gets the credit?" Is this intelligent behavior? Or was the proof lying deeply hidden within the human (Gelernter), and did the computer merely bring it to the surface? This last question comes close to hitting the mark. We can turn it around: Was the proof lying deeply hidden in the program? Or was it close to the surface? That is, how easy is it to see why the program did what it did? Can the discovery be attributed to some simple mechanism, or simple combination of mechanisms, in the program? Or was there a complex interaction which, if one heard it explained, would not diminish one's awe at its having happened?

It seems reasonable to say that if one can ascribe the performance to certain operations which are easily traced in the program, then in some sense the program was just revealing ideas which were in essence hiddenthough not too deeply-inside the programmer's own mind. Conversely, if

following the program does not serve to enlighten one as to why this particular discovery popped out, then perhaps one should begin to separate the program's "mind" from that of its programmer. The human gets credit for having invented the program, but not for having had inside his own head the ideas produced by the program. In such cases, the human can be referred to as the "meta-author"-the author of the author of the result, and the program as the (just plain) author.

In the particular case of Gelernter and his geometry machine, while Gelernter probably would not have rediscovered Pappus'. proof, still the mechanisms which generated that proof were sufficiently close to the surface of the program that one hesitates to call the program a geometer in its own right. If it had kept on astonishing people by coming up with ingenious new proofs over and over again, each of which seemed to be based on a fresh spark of genius rather than on some standard method, then one would have no qualms about calling the program a geometer-but this did not happen.

Who Composes Computer Music?

The distinction between author and meta-author is sharply pointed up in the case of computer composition of music. There are various levels of autonomy which a program may seem to have in the act of composition. One level is illustrated by a piece whose "meta-author" was Max Mathews of Bell Laboratories. He fed in the scores of the two marches "When Johnny Comes Marching Home" and "The British Grenadiers", and instructed the computer to make a new score-one which starts out as "Johnny", but slowly merges into "Grenadiers". Halfway through the piece, "Johnny" is totally gone, and one hears "Grenadiers" by itself ... Then the process is reversed, and the piece finishes with "Johnny", as it began. In Mathews' own words, this is

> ... a nauseating musical experience but one not without interest, particularly in the rhythmic conversions. "The Grenadiers" is written in 2/4 time in the key of F major. "Johnny" is written in 6/8 time in the key of E minor. The change from 2/4 to 6/8 time can be clearly appreciated, yet would be quite difficult for a human musician to play. The modulation from the key of F major to E minor, which involves a change of two notes in the scale, is jarring, and a smaller transition would undoubtedly have been a better *choice*."

The resulting piece has a somewhat droll quality to it, though in spots it is turgid and confused.

> Is the computer composing? The question is best unasked, but it cannot be completely ignored. An answer is difficult to provide. The algorithms are deterministic, simple, and understandable. No complicated or hard-to understand computations are involved; no "learning" programs are used; no random processes occur; the machine functions in a perfectly mechanical and straightforward manner. However, the result is sequences of sound that are unplanned in fine detail by the composer, even though the over-all structure

of the section is completely and precisely specified. Thus the composer is often surprised, and pleasantly surprised, ar the details of the realization of his ideas. To this extent only is the computer composing. We call the process algorithmic composition, but we immediately re-emphasize that the algorithms are transparently simple."

This is Mathews' answer to a question which he would rather "unask". Despite his disclaimer, however, many people find it easier to say simply that the piece was "composed by a computer". I believe this phrase misrepresents the situation totally. The program contained no structures analogous to the brain's "symbols", and could not be said in any sense to be "thinking" about what it was doing. To attribute the composition of such a piece of music to the computer would be like attributing the authorship of this book to the computerized automatically (often incorrectly) hyphenating phototypesetting machine with which it was set.

This brings up a question which is a slight digression from AI, but actually not a huge one. It is this: When you see the word "I" or "me" in a text, what do you take it to be referring to? For instance, think of the phrase "WASH ME" which appears occasionally on the back of dirty trucks. Who is this "me"? Is this an outcry of some forlorn child who, in desperation to have a bath, scribbled the words on the nearest surface? Or is the truck requesting a wash? Or, perhaps, does the sentence itself wish to be given a shower? Or, is it that the filthy English language is asking to be cleansed? One could go on and on in this game. In this case, the phrase is a joke, and one is supposed to pretend, on some level, that the truck itself wrote the phrase and is requesting a wash. On another level, one clearly recognizes the writing as that of a child, and enjoys the humor of the misdirection. Here, in fact, is a game based on reading the "me" at the wrong level.

Precisely this kind of ambiguity has arisen in this book, first in the Contracrostipunctus, and later in the discussions of Gödel's string G (and its relatives). The interpretation given for unplayable records was "I Cannot Be Played on Record Player X", and that for unprovable statements was, "I Cannot Be Proven in Formal System X". Let us take the latter sentence. On what other occasions, if any, have you encountered a sentence containing the pronoun "I" where you automatically understood that the reference was not to the speaker of the sentence, but rather to the sentence itself? Very few, I would guess. The word "I", when it appears in a Shakespeare sonnet, is referring not to a fourteen-line form of poetry printed on a page, but to a flesh-and-blood creature behind the scenes, somewhere off stage.

How far back do we ordinarily trace the "I" in a sentence? The answer, it seems to me, is that we look for a sentient being to attach the authorship to. But what is a sentient being? Something onto which we can map ourselves comfortably. In Weizenbaum's "Doctor" program, is there a personality? If so, whose is it? A small debate over this very question recently raged in the pages of Science magazine.

This brings us back to the issue of the "who" who composes computer music. In most circumstances, the driving force behind such pieces is a

human intellect, and the computer has been employed, with more or less ingenuity, as a tool for realizing an idea devised by the human. The program which carries this out is not anything which we can identify with. It is a simple and single-minded piece of software with no flexibility, no perspective on what it is doing, and no sense of self. If and when, however, people develop programs which have those attributes, and pieces of music start issuing forth from them, then I suggest that will be the appropriate, time to start splitting up one's admiration: some to the programmer for creating such an amazing program, and some to the program itself for its sense of music. And it seems to me that that will only take place when the internal structure of such a program is based on something similar to the "symbols" in our brains and their triggering patterns, which are responsible for the complex notion of meaning. The fact of having this kind of internal structure would endow the program with properties which would make us feel comfortable in identifying with it, to some extent. But until then, I will not feel comfortable in saying "this piece was composed by a computer".

Theorem Proving and Problem Reduction

Let us now return to the history of AI. One of the early things which people attempted to program was the intellectual activity of theorem proving. Conceptually, this is no different from programming a computer to look for a derivation of MU in the MIU-system, except that the formal systems involved were often more complicated than the MIU-system. They were versions of the Predicate Calculus, which is an extension of the Propositional Calculus involving quantifiers. Most of the rules of the Predicate Calculus are included in TNT, as a matter of fact. The trick in writing such a program is to instill a sense of direction, so that the program does not wander all over the map, but works only on "relevant" pathways-those which, by some reasonable criterion, seem to be leading towards the desired string.

In this book we have not dealt much with such issues. How indeed can you know when you are proceeding towards a theorem, and how can you tell if what you are doing is just empty fiddling? This was one thing which I hoped to illustrate with the MU-puzzle. Of course, there can be no definitive answer: that is the content of the limitative Theorems, since if you could always know which way to go, you could construct an algorithm for proving any desired theorem, and that would violate Church's Theorem. There is no such algorithm. (I will leave it to the reader to see exactly why this follows from Church's Theorem.) However, this doesn't mean that it is impossible to develop any intuition at all concerning what is and what is not a promising route; in fact, the best programs have very sophisticated heuristics, which enable them to make deductions in the Predicate Calculus at speeds which are comparable to those of capable humans.

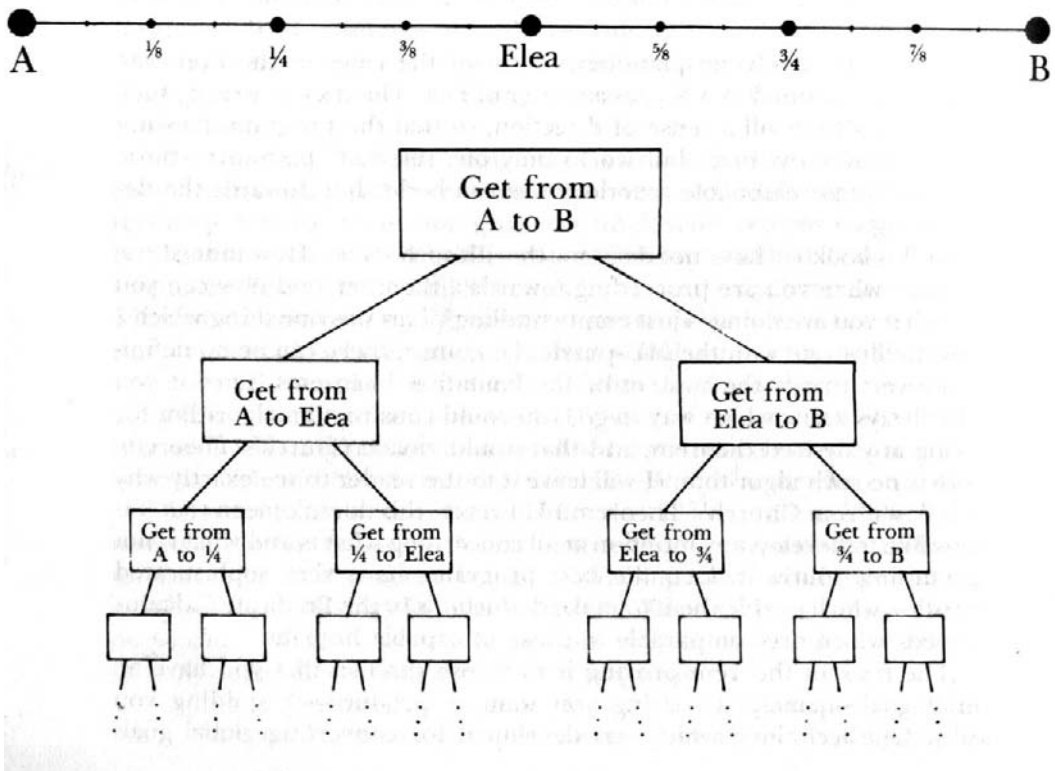
The trick in theorem proving is to use the fact that you have an overall goal-namely the string you want to produce-in guiding you locally. One technique which was developed for converting global goals

into local strategies for derivations is called problem reduction. It is based on the idea that whenever one has a long-range goal, there are usually subgoals whose attainment will aid in the attainment of the main goal. Therefore if one breaks up a given problem into a series of new subproblems, then breaks those in turn into subsubproblems, and so on, in a recursive fashion, one eventually comes down to very modest goals which can presumably be attained in a couple of steps. Or at least so it would seem ...

Problem reduction got Zeno into hot water. Zeno's method, you recall, for getting from A to B (think of B as the goal), is to "reduce" the problem into two subproblems: first go halfway, then go the rest of the way. So now you have "pushed"-in the sense of Chapter V-two subgoals onto your "goal stack". Each of these, in turn, will be replaced by two subsubgoals and so on ad infinitum. You wind up with an infinite goal-stack, instead of a single goal (Fig. 115). Popping an infinite number of goals off your stack will prove to be tricky-which is just Zeno's point, of course.

Another example of an infinite recursion in problem reduction occurred in the Dialogue Little Harmonic Labyrinth, when Achilles wanted to have a Typeless Wish granted. Its granting had to be deferred until permission was gotten from the Meta-Genie; but in order to get permission to give permission, she had to summon the Meta-Meta-Genie-and so on. Despite

FIGURE 115. *Zeno's endless goal tree, for getting from A to B.*



the infiniteness of the goal stack, Achilles got his wish. Problem reduction wins the day!

Despite my mockery, problem reduction is a powerful technique for converting global problems into local problems. It shines in certain situations, such as in the endgame of chess, where the look-ahead technique often performs miserably, even when it is carried to ridiculous lengths, such as fifteen or more plies. This is because the look-ahead technique is not based on planning; it simply has no goals and explores a huge number of pointless alternatives. Having a goal enables you to develop a strategy for the achievement of that goal, and this is a completely different philosophy from looking ahead mechanically. Of course, in the look-ahead technique, desirability or its absence is measured by the evaluation function for positions, and that incorporates indirectly a number of goals, principally that of not getting checkmated. But that is too indirect. Good chess players who play against look-ahead chess programs usually come away with the impression that their opponents are very weak in formulating plans or strategies.

Shandy and the Bone

There is no guarantee that the method of problem reduction will work. There are many situations where it flops. Consider this simple problem, for instance. You are a dog, and a human friend has just thrown your favorite bone over a wire fence into another yard. You can see your bone through the fence, just lying there in the grass-how luscious! There is an open gate in the fence about fifty feet away from the bone. What do you do? Some dogs will just run up to the_ fence, stand next to it, and bark; others will dash up to the open gate and double back to the lovely bone. Both dogs can be said to be exercising the problem reduction technique; however, they represent the problem in their minds in different ways, and this makes all the difference. The barking dog sees the subproblems as (1) running to the fence, (2) getting through it, and (3) running to the bone-but that second subproblem is a "toughie", whence the barking. The other dog sees the subproblems as (1) getting to the gate; (2) going through the gate; (3) running to the bone. Notice how everything depends on the way you represent the "problem space"-that is, on what you perceive as reducing the problem (forward motion towards the overall goal) and what you perceive as magnifying the problem (backward motion away from the goal).

Changing the Problem Space

Some dogs first try running directly towards the bone, and when they encounter the fence, something clicks inside their brain; soon they change course, and run over to the gate. These dogs realize that what on first

glance seemed as if it would increase the distance between the initial situation and the desired situation-namely, running away from the bone but towards the open gate-actually would decrease it. At first, they confuse physical distance with problem distance. Any motion away from the bone seems, by definition, a Bad Thing. But then-somehow-they realize that they can shift their perception of what will bring them "closer" to the bone. In a properly chosen abstract space, moving towards the gate is a trajectory bringing the dog closer to the bone! At every moment, the dog is getting "closer"-in the new sense-to the bone. Thus, the usefulness of problem reduction depends on how you represent your problem mentally. What in one space looks like a retreat can in another space look like a revolutionary step forward.

In ordinary life, we constantly face and solve variations on the dog and-bone problem. For instance, if one afternoon I decide to drive one hundred miles south, but am at my office and have ridden my bike to work, I have to make an extremely large number of moves in what are ostensibly "wrong" directions before I am actually on my way in car headed south. I have to leave my office, which means, say, heading east a few feet; then follow the hall in the building which heads north, then west. Then I ride my bike home, which involves excursions in all the directions of the compass; and I reach my home. A succession of short moves there eventually gets me into my car, and I am off. Not that I immediately drive due south. of course-I choose a route which may involve some excursions north. west, or east, with the aim of getting to the freeway as quickly as possible.

All of this doesn't feel paradoxical in the slightest; it is done without even any sense of amusement. The space in which physical backtracking is perceived as direct motion towards the goal is built so deeply into my mind that I don't even see any irony when I head north. The roads and hallways and so forth act as channels which I accept without much fight, so that part of the act of choosing how to perceive the situation involves just accepting what is imposed. But dogs in front of fences sometimes have a hard time doing that, especially when that bone is sitting there so close, staring them in the face, and looking so good. And when the problem space is just a shade more abstract than physical space, people are often just as lacking in insight about what to do as the barking dogs.

In some sense all problems are abstract versions of the dog-and-bone problem. Many problems are not in physical space but in some sort of conceptual space. When you realize that direct motion towards the goal in that space runs you into some sort of abstract "fence", you can do one of two things: (1) try moving away from the goal in some sort of random way, hoping that you may come upon a hidden "gate" through which you can pass and then reach your bone; or (2) try to find a new "space" in which you can represent the problem, and in which there is no abstract fence separating you from your goal-then you can proceed straight towards the goal in this new space. The first method may seem like the lazy way to go, and the second method may seem like a difficult and complicated way to go. And yet, solutions which involve restructuring the problem space more often

than not come as sudden flashes of insight rather than as products of a series of slow, deliberate thought processes. Probably these intuitive flashes come from the extreme core of intelligence-and, needless to say, their source is a closely protected secret of our jealous brains.

In any case, the trouble is not that problem reduction per se leads to failures; it is quite a sound technique. The problem is a deeper one: how do you choose a good internal representation for a problem? What kind of "space" do you see it in? What kinds of action reduce the "distance" between you and your goal in the space you have chosen? This can be expressed in mathematical language as the problem of hunting for an approprate metric (distance function) between states. You want to find a metric in which the distance between you and your goal is very small.

Now since this matter of choosing an internal representation is itself a type of problem-and a most tricky one, too-you might think of turning the technique of problem reduction back on it! To do so, you would have to have a way of representing a huge variety of abstract spaces, which is an exceedingly complex project. I am not aware of anyone's having tried anything along these lines. It may be just a theoretically appealing, amusing suggestion which is in fact wholly unrealistic. In any case, what Al sorely lacks is programs which can "step back" and take a look at what is going on, and with this perspective, reorient themselves to the task at hand. It is one thing to write a program which excels at a single task which, when done by a human being, seems to require intelligence-and it is another thing altogether to write an intelligent program! It is the difference between the Sphex wasp (see Chapter XI), whose wired-in routine gives the deceptive appearance of great intelligence, and a human being observing a Sphex wasp.

The I-Mode and the M-Mode Again

An intelligent program would presumably be one which is versatile enough to solve problems of many different sorts. It would learn to do each different one and would accumulate experience in doing so. It would be able to work within a set of rules and yet also, at appropriate moments, to step back and make a judgment about whether working within that set of rules is likely to be profitable in terms of some overall set of goals which it has. It would be able to choose to stop working within a given framework, if need be, and to create a new framework of rules within which to work for a while.

Much of this discussion may remind you of aspects of the **MU**-puzzle. For instance, moving away from the goal of a problem is reminiscent of moving away from **MU** by making longer and longer strings which you hope may in some indirect way enable you to make **MU**. If you are a naive "dog", you may feel you are moving away from your "**MU**-bone" whenever your string increases beyond two characters; if you are a more sophisticated dog, the use of such lengthening rules has an indirect justification, something like heading for the gate to get your **MU**-bone.

Another connection between the previous discussion and the **MU** puzzle is the two modes of operation which led to insight about the nature of the **MU**-puzzle: the Mechanical mode, and the Intelligent mode. In the former, you are embedded within some fixed framework; in the latter, you can always step back and gain an overview of things. Having an overview is tantamount to choosing a representation within which to work; and working within the rules of the system is tantamount to trying the technique of problem reduction within that selected framework. Hardy's comment on Ramanujan's style-particularly his willingness to modify his own hypotheses-illustrates this interplay between the **M**-mode and the **I**-mode in creative thought.

The Sphex wasp operates excellently in the **M**-mode, but it has absolutely no ability to choose its framework or even to alter its **M**-mode in the slightest. It has no ability to notice when the same thing occurs over and over and over again in its system, for to notice such a thing would be to jump out of the system, even if only ever so slightly. It simply does not notice the sameness of the repetitions. This idea (of not noticing the identity of certain repetitive events) is interesting when we apply it to ourselves. Are there highly repetitious situations which occur in our lives time and time again, and which we handle in the identical stupid way each time, because we don't have enough of an overview to perceive their sameness? This leads back to that recurrent issue, "What is sameness?" It will soon come up as an Al theme, when we discuss pattern recognition.

Applying Al to Mathematics

Mathematics is in some ways an extremely interesting domain to study from the Al point of view. Every mathematician has the sense that there is a kind of metric between ideas in mathematics-that all of mathematics is a network of results between which there are enormously many connections. In that network, some ideas are very closely linked; others require more elaborate pathways to be joined. Sometimes two theorems in mathematics are close because one can be proven easily, given the other. Other times two ideas are close because they are analogous, or even isomorphic. These are two different senses of the word "close" in the domain of mathematics. There are probably a number of others. Whether there is an objectivity or a universality to our sense of mathematical closeness, or whether it is largely an accident of historical development is hard to say. Some theorems of different branches of mathematics appear to us hard to link, and we might say that they are unrelated-but something might turn up later which forces us to change our minds. If we could instill our highly developed sense of mathematical closeness-a "mathematician's mental metric", so to speak-into a program, we could perhaps produce a primitive "artificial mathematician". But that depends on being able to convey a sense of simplicity or "naturalness" as well, which is another major stumbling block.

These issues have been confronted in a number of AI projects. There

is a collection of programs developed at **MIT** which go under the name «**MACSYMA**", whose purpose it is to aid mathematicians in symbolic manipulation of complex mathematical, expressions. This program has in it some sense of "where to go"-a sort of "complexity gradient" which guides it from what we would generally consider complex expressions to simpler ones. Part of **MACSYMA's** repertoire is a program called "**SIN**", which does symbolic integration of functions; it is generally acknowledged to be superior to humans in some categories. It relies upon a number of different skills, as intelligence in general must: a vast body of knowledge, the technique of problem reduction, a large number of heuristics, and also some special tricks.

Another program, written by Douglas Lenat at Stanford, had as its aim to invent concepts and discover facts in very elementary mathematics. Beginning with the notion of sets, and a collection of notions of what is "interesting" which had been spoon-fed into it, it "invented" the idea of counting, then the idea of addition, then multiplication, then-among other things-the notion of prime numbers, and it went so far as to rediscover Goldbach's conjecture! Of course these "discoveries" were all hundreds-even thousands-of years old. Perhaps this may be explained in part by saying that the sense of "interesting" was conveyed by Lenat in a large number of rules which may have been influenced by his twentieth century training; nonetheless it is impressive. The program seemed to run out of steam after this very respectable performance. An interesting thing about it was that it was unable to develop or improve upon its own sense of what is interesting. That seemed another level of difficulty up-or perhaps several levels up.

The Crux of Al: Representation of Knowledge

Many of the examples above have been cited in order to stress that the way a domain is represented has a huge bearing on how that domain is "understood". A program which merely printed out theorems of TNT in a preordained order would have no understanding of number theory; a program such as Lenat's with its extra layers of knowledge could be said to have a rudimentary sense of number theory; and one which embeds mathematical knowledge in a wide context of real-world experience would probably be the most able to "understand" in the sense that we think we do. It is this' *representation of knowledge* that is at the crux of Al.

In the early days it was assumed that knowledge came in sentence-like packets", and that the best way to implant knowledge into a program was to develop a simple way of translating facts into small passive packets of data. Then every fact would simply be a piece of data, accessible to the programs using it. This is exemplified by chess programs, where board Positions are coded into matrices or lists of some sort and stored efficiently in memory where they can be retrieved and acted upon by subroutines.

The fact that human beings store facts in a more complicated way was

Known to psychologists for quite a while and has only recently been rediscovered by AI workers, who are now confronting the problems of "chunked" knowledge, and the difference between procedural and declarative types of knowledge, which is related, as we saw in Chapter XI, to the difference between knowledge which is accessible to introspection and knowledge which is inaccessible to introspection.

The naive assumption that all knowledge should be coded into passive pieces of data is actually contradicted by the most fundamental fact about computer design: that is, how to add, subtract, multiply, and so on is not coded into pieces of data and stored in memory; it is, in fact, represented nowhere in memory, but rather in the wiring patterns of the hardware. A pocket calculator does not store in its memory knowledge of how to add; that knowledge is encoded into its "guts". There is no memory location to point to if somebody demands, "Show me where the knowledge of how to add resides in this machine!"

A large amount of work in Al has nevertheless gone into systems in which the bulk of the knowledge is stored in specific places-that is, declaratively. It goes without saying that some knowledge has to be embodied in programs; otherwise one would not have a program at all, but merely an encyclopedia. The question is how to split up knowledge between program and data. Not that it is always easy to distinguish between program and data, by any means. I hope that was made clear enough in Chapter XVI. But in the development of a system, if the programmer intuitively conceives of some particular item as data (or as program), that may have significant repercussions on the system's structure, because as one programs one does tend to distinguish between data-like objects and program-like objects.

It is important to point out that in principle, any manner of coding information into data structures or procedures is as good as any other, in the sense that if you are not too concerned about efficiency, what you can do in one scheme, you can do in the other. However, reasons can be given which seem to indicate that one method is definitely superior to the other. For instance, consider the following argument in favor of using procedural representations only: "As soon as you try to encode features of sufficient complexity into data, you are forced into developing what amounts to a new language, or formalism. So in effect your data structures become program-like, with some piece of your program serving as their interpreter; you might as well represent the same information directly in procedural form to begin with, and obviate the extra level of interpretation."

DNA and Proteins Help Give Some Perspective

This argument sounds quite convincing, and yet, if interpreted a little loosely, it can be read as an argument for the abolishment of **DNA** and **RNA**. Why encode genetic information in **DNA**, when by representing it directly in proteins, you could eliminate not just one, but *two* levels of interpretation? The answer is: it turns out that it is extremely useful to have

the same information in several different forms for different purposes. One advantage of storing genetic information in the modular and data-like form of **DNA** is that two individuals' genes can be easily recombined to form a new genotype. This would be very difficult if the information were only in proteins. A second reason for storing information in **DNA** is that it is easy to transcribe and translate it into proteins. When it is not needed, it does not take up much room; when it is needed, it serves as a template. There is no mechanism for copying one protein off of another; their folded tertiary structures would make copying highly unwieldy. Complementarily, it is almost imperative to be able to get genetic information into three-dimensional structures such as enzymes, because the recognition and manipulation of molecules is by its nature a three-dimensional operation. Thus the argument for purely procedural representations is seen to be quite fallacious in the context of cells. It suggests that there are advantages to being able to switch back and forth between procedural and declarative representations. This is probably true also in AI.

This issue was raised by Francis Crick in a conference on communication with extraterrestrial intelligence:

> We see on Earth that there are two molecules, one of which is good for replication [**DNA**] and one of which is good for action [proteins]. Is it possible to devise a system in which one molecule does both jobs, or are there perhaps strong arguments, from systems analysis, which might suggest (if they exist) that to divide the job into two gives a great advantage, This is a question to which I do not know the answer.[14]

Modularity of Knowledge

Another question which comes up in the representation of knowledge is modularity. How easy is it to insert new knowledge? How easy is it to revise old knowledge? How modular are books? It all depends. If from a tightly structured book with many cross-references a single chapter is removed, the rest of the book may become virtually incomprehensible. It is like trying to pull a single strand out of a spider web-you ruin the whole in doing so. On the other hand, some books are quite modular, having independent chapters.

Consider a straightforward theorem-generating program which uses **TNT**'s axioms and rules of inference. The "knowledge" of such a program has two aspects. It resides implicitly in the axioms and rules, and explicitly in the body of theorems which have so far been produced. Depending on which way you look at the knowledge, you will see it either as modular or as spread all around and completely nonmodular. For instance, suppose you had written such a program but had forgotten to include **TNT**'s Axiom I in the list of axioms. After the program had done many thousands of derivations, you realized your oversight, and inserted the new axiom. The fact that you can do so in a trice shows that the system's implicit knowledge is modular; but the new axiom's contribution to the explicit knowledge of the system will only be reflected after a long time-after its effects have "dif-

fused" outwards, as the odor of perfume slowly diffuses in a room when the bottle is broken. In that sense the new knowledge takes a long time to be incorporated. Furthermore, if you wanted to go back and replace Axiom I by its negation, you could not just do that by itself; you would have to delete all theorems which had involved Axiom 1 in their derivations. Clearly this system's explicit knowledge is not nearly so modular as its implicit knowledge.

It would be useful if we learned how to transplant knowledge modularly. Then to teach everyone French, we would just open up their heads and operate in a fixed way on their neural structures-then they would know how to speak French. Of course, this is only a hilarious pipe dream.

Another aspect of knowledge representation has to do with the way in which one wishes to use the knowledge. Are inferences supposed to be drawn as pieces of information arrive? Should analogies and comparisons constantly be being made between new information and old information? In a chess program, for instance, if you want to generate look-ahead trees, then a representation which encodes board positions with a minimum of redundancy will be preferable to one which repeats the information in several different ways. But if you want your program to "understand" a board position by looking for patterns and comparing them to known patterns, then representing the same information several times over in different forms will be more useful.

Representing Knowledge in a Logical Formalism

There are various schools of thought concerning the best way to represent and manipulate knowledge. One which has had great influence advocates representations using formal notations similar to those for **TNT**-using propositional connectives and quantifiers. The basic operations in such representations are, not surprisingly, formalizations of deductive reasoning. Logical deductions can be made using rules of inference analogous to some of those in **TNT**. Querying the system about some particular idea sets up a goal in the form of a string to be derived. For example: "Is **MUMON** a theorem?" Then the automatic reasoning mechanisms take over in a goal oriented way, using various methods of problem reduction.

For example, suppose that the proposition "All formal arithmetics are incomplete" were known, and the program were queried, "Is *Principia Mathematica* incomplete?" In scanning the list of known facts-often called the data base-the system might notice that if it could establish that *Principia Mathematica* is a formal arithmetic, then it could answer the question. Therefore the proposition "*Principia Mathematica* is a formal arithmetic" would be set up as a subgoal, and then problem reduction would take over. If it could find further things which would help in establishing (or refuting) the goal or the subgoal, it would work on them-and so on, recursively. This process is given the name of *backwards chaining*, since it begins with the goal and works its way backwards, presumably towards things which may already be known. If one makes a graphic representation of the main goal,

subsidiary goals, subsubgoals, etc., a tree-like structure will arise, since the main goal may involve several different subgoals, each of which in turn involves several subsubgoals, etc.

Notice that this method is not guaranteed to resolve the question, for there may be no way of establishing within the system that Principia Mathematica is a formal arithmetic. This does not imply, however, that either the goal or the subgoal is a false statement-merely that they cannot be derived with the knowledge currently available to the system. The system may print out, in such a circumstance, "I do not know" or words to that effect. The fact that some questions are left open is of course similar to the incompleteness from which certain well-known formal systems suffer.

Deductive vs. Analogical Awareness

This method affords a deductive awareness of the domain that is represented, in that correct logical conclusions can be drawn from known facts. However, it misses something of the human ability to spot similarities and to compare situations-it misses what might be called analogical awareness-a crucial side of human intelligence. This is not to say that analogical thought processes cannot be forced into such a mold, but they do not lend themselves naturally to being captured in that kind of formalism. These days, logic-oriented systems are not so much in vogue as other kinds, which allow complex forms of comparisons to be carried out rather naturally.

When you realize that knowledge representation is an altogether different ball game than mere storage of numbers, then the idea that "a computer has the memory of an elephant" is an easy myth to explode. What is *stored in memory* is not necessarily synonymous with what a program *knows*; for even if a given piece of knowledge is encoded somewhere inside a complex system, there may be no procedure, or rule, or other type of handler of data, which can get at it-it may be inaccessible. In such a case, you can say that the piece of knowledge has been "forgotten" because access to it has been temporarily or permanently lost. Thus a computer program may "forget" something on a high level which it "remembers" on a low level. This is another one of those ever-recurring level distinctions, from which we can probably learn much about our own selves. When a human forgets, it most likely means that a high-level pointer has been lost-not that any information has been deleted or destroyed. This highlights the extreme importance of keeping track of the ways in which you store incoming experiences, for you never know in advance under what circumstances, or from what angle, you will want to pull something out of storage.

From Computer Haiku to an RTN-Grammar

The complexity of the knowledge representation in human heads first hit home with me when I was working on a program to generate English sentences "out of the blue". I had come to this project in a rather interest-

ing way. I had heard on the radio a few examples of so-called "Computer Haiku". Something about them struck me deeply. There was a large element of humor and simultaneously mystery to making a computer generate something which ordinarily would be considered an artistic creation. I was highly amused by the humorous aspect, and I was very motivated by the mystery-even contradiction-of programming creative acts. So I set out to write a program even more mysteriously contradictory and humorous than the haiku program.

At first I was concerned with making the grammar flexible and recursive, so that one would not have the sense that the program was merely filling in the blanks in some template. At about that time I ran across a *Scientific American* article by Victor Yngve in which he described a simple but flexible grammar which could produce a wide variety of sentences of the type found in some children's books. I modified some of the ideas I'd gleaned from that article and came up with a set of procedures which formed a Recursive Transition Network grammar, as described in Chapter V. In this grammar, the selection of words in a sentence was determined by a process which began by selecting-at random-the overall structure of the sentence; gradually the decision-making process trickled down through lower levels of structure until the word level and the letter level were reached. A lot had to be done below the word level, such as inflecting verbs and making plurals of nouns; also irregular verb and noun forms were first formed regularly, and then if they matched entries in a table, substitutions of the proper (irregular) forms were made. As each word reached its final form, it was printed out. The program was like the proverbial monkey at a typewriter, but operating on several levels of linguistic structure simultaneously-not just the letter level.

In the early stages of developing the program, I used a totally silly vocabulary-deliberately, since I was aiming at humor. It produced a lot of nonsense sentences, some of which had very complicated structures, others of which were rather short. Some excerpts are shown below:

> A male pencil who must laugh clumsily would quack. Must program not always crunch girl at memory? The decimal bug which spits clumsily might tumble. Cake who does sure take an unexpected man within relationship might always dump card.
> Program ought run cheerfully.
> The worthy machine ought not always paste the astronomer.
> Oh, program who ought really run off of the girl writes musician for theater. The businesslike relationship quacks.
> The lucky girl which can always quack will never sure quack.
> The game quacks. Professor will write pickle. A bug tumbles. Man takes the box who slips.

The effect is strongly surrealistic and at times a little reminiscent of

haiku-for example. the final sample of four consecutive short sentences. At first it seemed very funny and had a certain charm, but soon it became rather stale. After reading a few pages of output one could sense the limits of the space in which the program was operating; and after that, seeing random points inside that space-even though each one was "new"-was nothing new. This is, it seems to me, a general principle: you get bored with something not when you have exhausted its repertoire of behavior, but when you have mapped out the limits of the space that contains its behavior. The behavior space of a person is just about complex enough that it can continually surprise other people; but that wasn't true of my program. I realized that my goal of producing truly humorous output would require that far more subtlety be programmed in. But what, in this case, was meant by "subtlety It was clear that absurd juxtapositions of words were just too unsubtle; I needed a way to ensure that words would be used in accordance with the realities of the world. This was where thoughts about representation of knowledge began to enter the picture.

From RTN's to ATN's

The idea I adopted was to classify each word-noun, verb, preposition, etc.-in several different "semantic dimensions". Thus, each word was a member of classes of various sorts; then there were also superclasses-classes of classes (reminiscent of the remark by Ulam). In principle, such aggregation could continue to any number of levels, but I stopped at two. At any given moment, the choice of words was now semantically restricted, because it was required that there should be *agreement* between the various parts of the phrase being constructed. The idea was, for instance, that certain kinds of acts could be performed only by animate objects; that only certain kinds of abstractions could influence events, and so on. The decisions about what categories were reasonable, and whether each category was better thought of as a class or a superclass, were quite complicated. All words were branded in several different dimensions. Common prepositions-"of", "in", etc.-had several distinct entries, corresponding to their distinct usages. Now, the output began to be much more comprehensible-and for that reason it was funny in a new way.

A Little Turing Test

Below, I have reproduced nine selections, carefully culled from many pages of output from later versions of my program. Along with them are three (seriously intended) human-written sentences. Which?

(1) Blurting may be considered as the reciprocal substitution of semiotic material (dubbing) for a semiotic dialogical product in a dynamic reflexion.

(2) Rather think of a pathway of a 'sequence' of gedankenexperiment simpletons where heir-lines are a prima facie case of a paradiachronic transitivity.

(3) Think of that as a chain strength possibility of what, eventually, comes out as a product (epistemic conditions:,) and the product is not a Frankfurt-ish packing-it-all-in.

(4) Despite the efforts, the reply, if you will, had been supported by the Orient; hence a fallacy will thereafter be suspended by the attitude which will be being held by the ambassador.

(5) Of course, until the upheavals, the ambassador was slightly gradually mollycoddling the rabble.

(6) Supposedly, refined liberty caused the attitudes insofar as peace is distilled by the consequences which will not eventually be caused by the command irrevocably insofar as peace of it is sometimes causing the intransigency infinitesimally surprisingly.

(7) According to the sophists, the campaigns in the city-states, in other words, have been accepted by the Orient cunningly. Of course, the Orient has been separated by the states particularly violently.
The Orient supports the efforts which had been supported by mankind.

(8) Admittedly, the hierarchical origin of the fallacy, nevertheless, will be prophesied by the enemies of it. By the same token, the individualists will have testified that intransigency will not have suspended the campaigns.

(9) Needless to say, during the upheaval which will have warranted the secrecy, the replies do not separate the Orient. Of course, the countries, ipso facto, are always probing liberty.

(10) Although a Nobel Prize was being achieved by the humanists, yet in addition, it was being achieved by the serf.

(11) An attitude will often be held by the serfs of a strife-torn nation.

(12) Moreover, the Nobel Prizes will be achieved. By the same token, despite the consequence, the Nobel Prizes which will be achieved will sometimes be achieved by a woman.

The human-written sentences are numbers 1 to 3; they were drawn from the contemporary journal Art-Language15 and are-as far as I can tellcompletely serious efforts among literate and sane people to communicate something to each other. That they appear here out of context is not too misleading, since their proper context sounds just the same as they do.

My program produced the rest. Numbers 10 to 12 were chosen to show that there were occasional bursts of total lucidity; numbers 7 to 9 are more typical of the output, floating, in that curious and provocative netherworld between meaning and no-meaning; and then numbers 4 to 6 pretty much transcend meaning. In a generous mood, one could say that they stand on their own as pure "language objects", something like pieces of abstract sculpture carved out of words instead of stone; alternatively, one could say that they are pure pseudointellectual drivel.

My choice of vocabulary was still aimed at producing humorous effects. The flavor of the output is hard to characterize. Although much of it "makes sense", at least on a single-sentence level, one definitely gets the feeling that the output is coming from a source with no understanding of what it is saying and no reason to say it. In particular, one senses an utter lack of visual imagery behind the words. When I saw such sentences come pouring out of the line printer, I experienced complex emotions. I was very amused by the silliness of the output. I was also very proud of my achievement and tried to describe it to friends as similar to giving rules for building up meaningful stories in Arabic out of single strokes of the pen-an exaggeration, but it pleased me to think of it that way. And lastly I was deeply thrilled by the knowledge that this enormously complicated machine was shunting around long trains of symbols inside it according to rules, and that these long trains of symbols were something like thoughts in my own head ... something like them.

Images of What Thought Is

Of course I didn't fool myself into thinking that there was a conscious being behind those sentences-far from it. Of all people, I was the most aware of the reasons that this program was terribly remote from real thought. Tesler's Theorem is quite apt here: as soon as this level of language handling ability had been mechanized, it was clear that it did not constitute intelligence. But this strong experience left me with an image: a glimmering sense that ***real*** thought was composed of much longer, much more complicated trains of symbols in the brain-many trains moving simultaneously down many parallel and crisscrossing tracks, their cars being pushed and pulled, attached and detached, switched from track to track by a myriad neural shunting-engines ...

It was an intangible image which I cannot convey in words, and it was only an image. But images and intuitions and motivations lie mingled close in the mind, and my utter fascination with this image was a constant spur to think more deeply about what thought really could be. I have tried in other parts of this book to communicate some of the daughter images of this original image-particularly in the *Prelude, Ant Fugue*.

What stands out in my mind now, as I look back at this program from the perspective of a dozen years, is how there is no sense of imagery behind what is being said. The program had no idea what a serf is, what a person is, or what anything at all is. The words were empty formal symbols, as empty

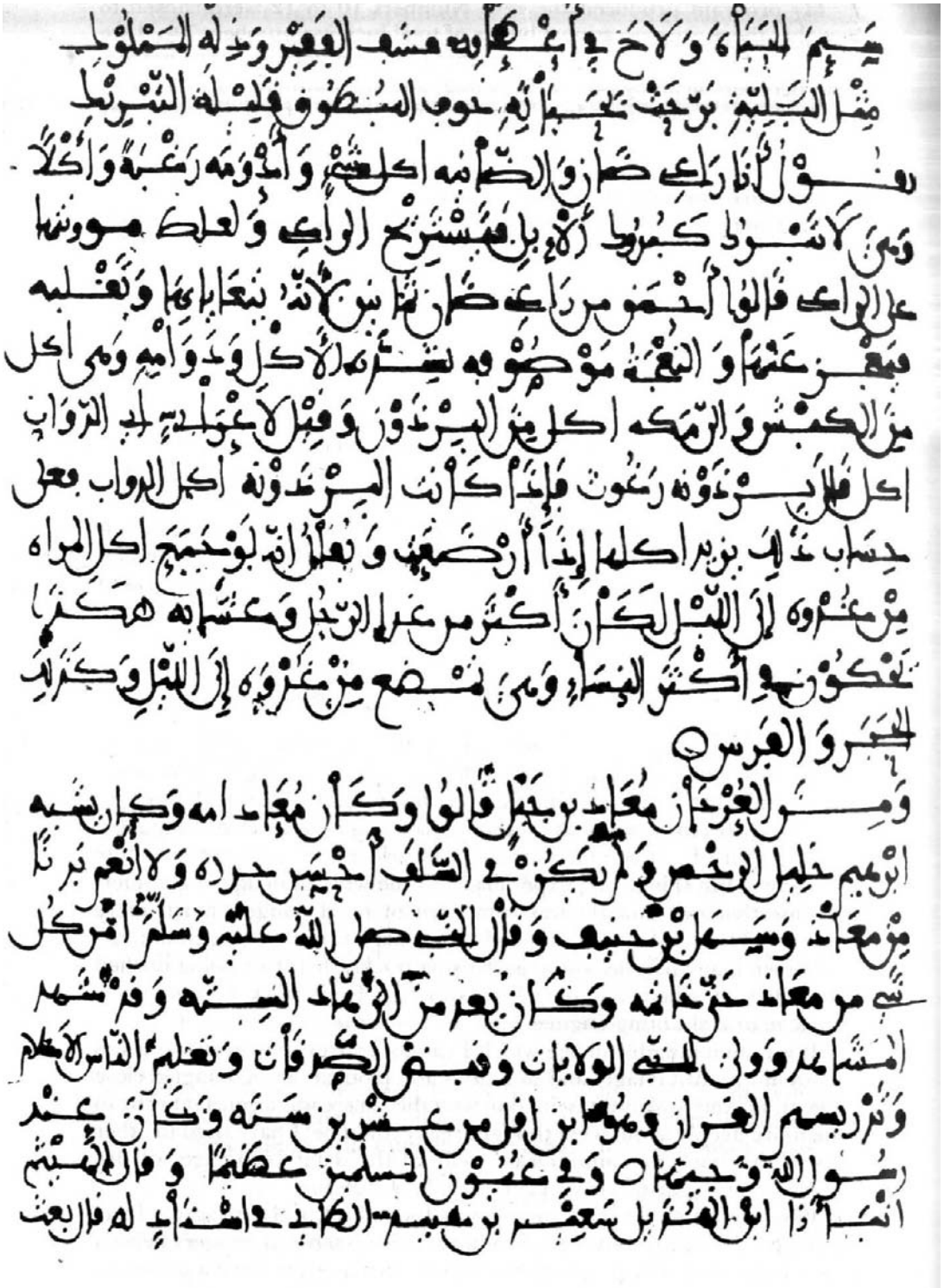


FIGURE 116. *A meaningful story in Arabic. [From A. Khatibi and M. S~elmassi, The Splendour of Islamic Calligraphy (New York: Rizzoli, 1976).*

as-perhaps emptier than-the p and q of the pq-system. My program took advantage of the fact that when people read text, they quite naturally tend to imbue each word with its full flavor-as if that were necessarily attached to the group of letters which form the word. My program could be looked at as a formal system, whose "theorems"-the output sentences-had ready-made interpretations (at least to speakers of English). But unlike the pq-system, these "theorems" were not all true statements when interpreted that way. Many were false, many were nonsense.

In its humble way, the pq-system mirrored a tiny corner of the world. But when my program ran, there was no mirror inside it of how the world works, except for the small semantic constraints which it had to follow. To create such a mirror of understanding, I would have had to wrap each concept in layers and layers of knowledge about the world. To do this would have been another kind of effort from what I had intended to do. Not that I didn't often think of trying to do it-but I never got around to trying it out.

Higher-Level Grammars ...

In fact, I often pondered whether I could write an **ATN**-grammar (or some other kind of sentence-producing program) which would only produce true sentences about the world. Such a grammar would imbue the words with genuine meanings, in the way it happened in the pq-system and in **TNT**. This idea of a language in which false statements are ungrammatical is an old one, going back to Johann Amos Comenius, in 1633. It is very appealing because you have a crystal ball embodied in your grammar: just write down the statement you want to know about, and check to see if it is grammatical... . Actually, Comenius went even further, for in his language, false statements were not only ungrammatical-they were inexpressible!

Carrying this thought in another direction, you might imagine a highlevel grammar which would produce random koans. Why not? Such a grammar would be equivalent to a formal system whose theorems are koans. And if you had such a program, could you not arrange it to produce only genuine koans? My friend Marsha Meredith was enthusastic about this idea of "Artificial Ism", so she tackled the project of writing a koan-writing program. One of her early efforts produced this curious quasi-koan:

> A SMALL YOUNG MASTER WANTED A SMALL WHITE GNARLED BOWL. "HOW CAN WE LEARN AND UNDERSTAND WITHOUT STUDY" THE YOUNG MASTER ASKED A LARGE CONFUSED MASTER. THE CONFUSED MASTER WALKED FROM A BROWN HARD MOUNTAIN TO A WHITE SOFT MOUNTAIN WITH A SMALL RED STONY BOWL. THE CONFUSED MASTER SAW A RED SOFT HUT.THE CONFUSED MASTER WANTED THE HUT. "WHY DID BODHIDHARMA COME INTO CHINA?" THE CONFUSED MASTER

ASKED A LARGE ENLIGHTENED STUDENT. "THE PEACHES ARE LARGE", THE STUDENT ANSWERED THE CONFUSED MASTER.w "HOW CAN WE LEARN AND UNDERSTAND WITHOUT STUDY?" THE CONFUSED MASTER ASKED A LARGE OLD MASTER. THE OLD MASTER WALKED FROM A WHITE STONY G0025. THE OLD MASTER GOT LOST.

Your personal decision procedure for koan genuineness probably reached a verdict without need of the Geometric Code or the Art of Zen Strings. If the lack of pronouns or the unsophisticated syntax didn't arouse your suspicions, that strange "00025" towards the end must have. What is it? It is a strange fluke-a manifestation of a bug which caused the program to print out, in place of the English word for an object, the program's internal name for the "node" (a **LISP** atom, in fact) where all information concerning that particular object was stored. So here we have a "window" onto a lower level of the underlying Zen mind-a level that should have remained invisible. Unfortunately, we don't have such clear windows onto the lower levels of human Zen minds.

The sequence of actions, though a little arbitrary, comes from a recursive **LISP** procedure called "**CASCADE**", which creates chains of actions linked in a vaguely causal way to each other. Although the degree of comprehension of the world possessed by this koan generator is clearly not stupendous, work is in progress to make its output a little more genuine seeming.

Grammars for Music?

Then there is music. This is a domain which you might suppose, on first thought, would lend itself admirably to being codified in an **ATN** grammar, or some such program. Whereas (to continue this naive line of thought) language relies on connections with the outside world for meaning, music is purely formal. There is no reference to things "out there" in the sounds of music; there is just pure syntax-note following note, chord following chord, measure following measure, phrase following phrase...

But wait. Something is wrong in this analysis. Why is some music so much deeper and more beautiful than other music? It is because form, in music, is expressive-expressive to some strange subconscious regions of our minds. The sounds of music do not refer to serfs or city-states, but they do trigger clouds of emotion in our innermost selves; in that sense musical meaning is dependent on intangible links from the symbols to things in the world-those "things", in this case, being secret software structures in our minds. No, great music will not come out of such an easy formalism as an **ATN**-grammar. Pseudomusic, like pseudo-fairy tales, may well come out-and that will be a valuable exploration for people to make-but the secrets of meaning in music lie far, far deeper than pure syntax.

I should clarify one point here: in principle, ATN-grammars have all the power of any programming formalism, so if musical meaning is captur-

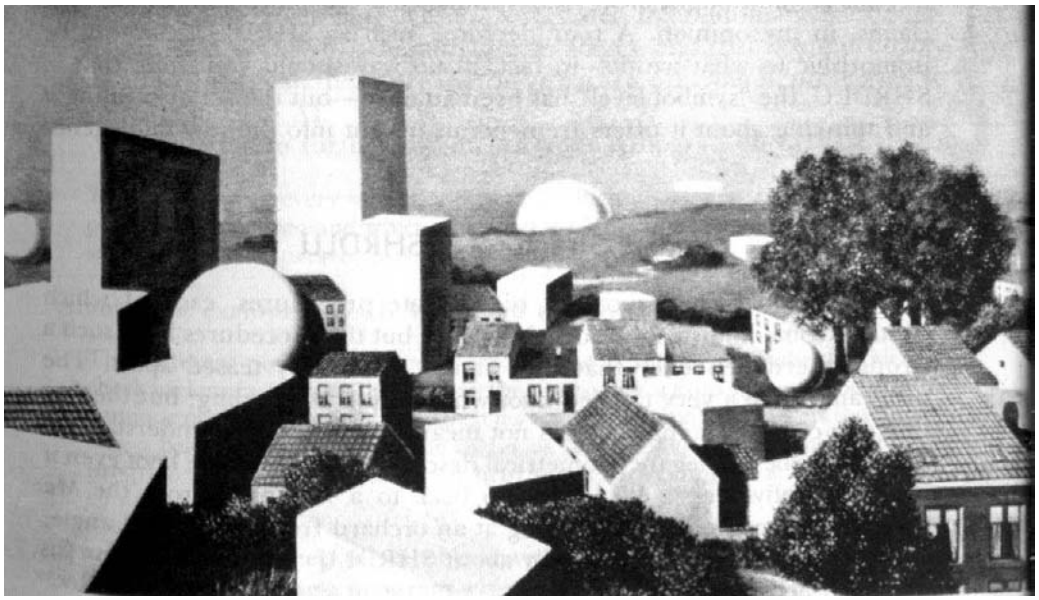
able in any way at all (which I believe it is), it is capturable in an A I N - grammar. True. But in that case, I maintain, the grammar will be defining not just musical structures, but the entire structures of the mind of a beholder. The "grammar" will be a full grammar of thought-not just a grammar of music.

Winograd's Program SHRDLU

What kind of program would it take to make human beings admit that it had some "understanding", even if begrudgingly? What would it take before you wouldn't feel intuitively that there is "nothing there"?

In the years 1968-70, Terry Winograd (alias Dr. Tony Earrwig) was a doctoral student at **MIT**, working on the joint problems of language and understanding. At that time at **MIT**, much Al research involved the so-called *blocks world*-a relatively simple domain in which problems concerning both vision and language-handling by computer could fit easily. The blocks world consists of a table with various kinds of toy-like blocks on it-square ones, oblong ones, triangular ones, etc., in various colors. (For a "blocks world" of another kind, see Figure 117: the painting *Mental Arithmetic* by Magritte. I find its title singularly appropriate in this context.) The vision problems in the **MIT** blocks world are very tricky: how can a computer figure out, from a TV-scan of a scene with many blocks in it, just what kinds of blocks are present, and what their relationships are? Some blocks may be perched on top of others, some may be in front of others, there may be shadows, and so on.

FIGURE 117. *Mental Arithmetic, by Rene Magritte (1931).*



Winograd s work was separate from the issues of vision, however. Beginning with the assumption that the blocks world was well represented h inside the computer's memory, he confronted the many-faceted problem of how to get the computer to:

(1) understand questions in English about the situation;
(2) give answers in English to questions about the situation;
(3) understand requests in English to manipulate the blocks;
(4) break down each request into a sequence of operations it could do;
(5) understand what it had done, and for what reasons;
(6) describe its actions and their reasons, in English.

It might seem reasonable to break up the overall program into modular subprograms, with one module for each different part of the problem; then, after the modules have been developed separately, to integrate them smoothly. Winograd found that this strategy of developing independent modules posed fundamental difficulties. He developed a radical approach, which challenged the theory that intelligence can be compartmentalized into independent or semi-independent pieces. His program **SHRDLU** named after the old code "**ETAOIN SHRDLU**", used by linotype operators to mark typos in a newspaper column-did not separate the problem into clean conceptual parts. The operations of parsing sentences, producing internal representations, reasoning about the world represented inside itself, answering questions, and so on, were all deeply and intricately meshed together in a procedural representation of knowledge. Some critics have charged that his program is so tangled that it does not represent any "theory" at all about language, nor does it contribute in any way to our insights about thought processes. Nothing could be more wrong than such claims, in my opinion. A tour de force such as **SHRDLU** may not be isomorphic to what we do-in fact, in no way should you think that in **SHRDLU**, the "symbol level" has been attained-but the act of creating it and thinking about it offers tremendous insight into the way intelligence works.

The Structure of SHRDLU

In fact, **SHRDLU** does consist of separate procedures, each of which contains some knowledge about the world; but the procedures have such a strong interdependency that they cannot be cleanly teased apart. The program is like a very tangled knot which resists untangling; but the fact that you cannot untangle it does not mean that you cannot understand it. There may be an elegant geometrical description of the entire knot even if it is physically messy. We could go back to a metaphor from the Mu Offering, and compare it to looking at an orchard from a "natural" angle.

Winograd has written lucidly about **SHRDLU**. I quote here from his article in Schank and Colby's book:

> One of the basic viewpoints underlying the model is that all language use can be thought of as a way of activating procedures within the hearer. We can think of any utterance as a program-one that indirectly causes a set of operations to he carried out within the hearer's cognitive system. This "program writing" is indirect in the sense that we are dealing with an intelligent interpreter, who may take a set of actions which are quite different from those the speaker intended. The exact form is determined by his knowledge of the world, his expectations about the person talking to him, etc. in this program we have a simple version of this process of interpretation as it takes place in the robot. Each sentence interpreted by the robot is converted to a set of instructions in **PLANNER**. The program that is created is then executed to achieve the desired effect."

PLANNER Facilitates Problem Reduction

The language **PLANNER**, referred to here, is an At language whose principal feature is that some of the operations necessary for problem reduction are built in-namely, the recursive process of creating a tree of jsubgoals, subsubgoals, etc. What this means is that such processes, instead of having to be spelled out time and time again by the programmer, are automatically implied by so-called **GOAL**-statements. Someone who reads a **PLANNER** program will see no explicit reference to such operations; in argon, they are user-transparent. If one path-in the tree fails to achieve the desired goal, then the **PLANNER** program will "backtrack" and try another route. "Backtracking" is the magic word as far as **PLANNER** is concerned.

Winograd's program made excellent use of these features of **PLANNER**-more exactly, of **MICROPLANNER**, a partial implementation of the plans for **PLANNER**. In the past few years, however, people with the goal of developing At have concluded that automatic backtrack ing, as in **PLANNER**, has definite disadvantages, and that it will probably not lead to their goal; therefore they have backed off from it, preferring to try other routes to **AI**.

Let us listen to further comments from Winograd on **SHRDLU**:

> The definition of every word is a program which is called at an appropriate point in the analysis, and which can do arbitrary computations involving the sentence and the present physical situation."

Among the examples which Winograd cites is the following:

> The different possibilities for the meaning of "the" are procedures which check various facts about the context, then prescribe actions such as "Look for a unique object in the data base which fits this description", or "Assert that the object being described is unique as far as the speaker is concerned." The program incorporates a variety of heuristics for deciding what part of the context is relevant. [18]

It is amazing how deep this problem with the word "the" is. It is probably safe to say that writing a program which can fully handle the top five words

of English-"the", "of", "and", "a", and "to"-would be equivalent to solving the entire problem of AI, and hence tantamount to knowing what intelligence and consciousness are. A small digression: the five most common nouns in English are-according to the Word Frequency Book compiled by John B. Carroll et al-"time", "people", "way", "water", and "words" (in that order). The amazing thing about this is that most people have no idea that we think in such abstract terms. Ask your friends, and 10 to 1 they'll guess such words as "man", "house", "car", "dog", and "money". And while we're on the subject of frequencies-the top twelve letters in English, in order, according to Mergenthaler, are: "**ETAOIN SHRDLU**".

One amusing feature of **SHRDLU** which runs totally against the stereotype of computers as "number crunchers" is this fact, pointed out by Winograd: "Our system does not accept numbers in numeric form, and has only been taught to count to ten."19 With all its mathematical underpinning, SHRDLU is a mathematical ignoramus! Just like Aunt Hillary, **SHRDLU** doesn't know anything about the lower levels which make it up. Its knowledge is largely *procedural* (see particularly the remark by "Dr, Tony Earrwig" in section 11 of the previous Dialogue).

It is interesting to contrast the procedural embedding of knowledge in **SHRDLU** with the knowledge in my sentence-generation program. All of the syntactical knowledge in my program was procedurally embedded in Augmented Transition Networks, written in the language Algol; but the semantic knowledge-the information about semantic class membership-was static: it was contained in a short list of numbers after each word. There were a few words, such as the auxiliary verbs "to be", "to have", and others, which were represented totally in procedures in Algol, but they were the exceptions. By contrast, in **SHRDLU**, all words were represented as programs. Here is a case which demonstrates that, despite the theoretical equivalence of data and programs, in practice the choice of one over the other has major consequences.

Syntax and Semantics

And now, a few more words from Winograd:

> Our program does not operate by first parsing a sentence, then doing semantic analysis, and finally by using deduction to produce a response. These three activities go on concurrently throughout the understanding of a sentence. As soon as a piece of syntactic structure begins to take shape, a semantic program is called to see whether it might make sense, and the resultant answer can direct the parsing. In deciding whether it makes sense, the semantic routine may call deductive processes and ask questions about the real world. As an example, in sentence 34 of the Dialogue ("Put the blue pyramid on the block in the box"), the parser first comes up with "the blue pyramid on the block" as a candidate for a noun group. At this point, semantic analysis is done, and since "the" is definite, a check is made in the data base for the object being referred to. When no such object is found, the parsing is redirected to find the noun group "the blue pyramid". It will then go on to find "on the block

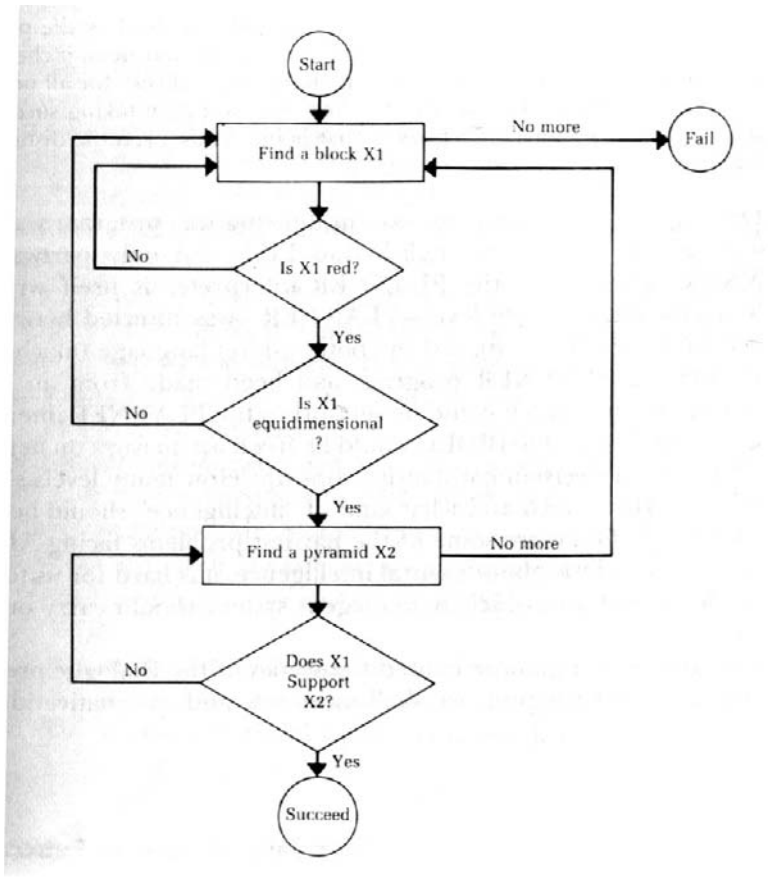
in the box" as a single phrase indicating a location .-.. Thus there is a continuing interplay between the different sorts of analysis, with the results of one affecting the others .[20]

It is extremely interesting that in natural language, syntax and semantics are so deeply intertwined. Last Chapter, in discussing the elusive concept of "form", we had broken the notion into two categories: syntactic form, which is detectable by a predictably terminating decision procedure, and semantic form, which is not. But here, Winograd is telling us that-at least when the usual senses of "syntax" and "semantics" are taken-they merge right into each other, in natural language. The external form of a sentence-that is, its composition in terms of elementary signs-does not divide up so neatly into syntactic and semantic aspects. This significant point for linguistics.

Here are some final comments on **SHRDLU** by Winograd.

Let us look at what the system would do with a simple description like "a red cube which supports a pyramid". The description will use concepts like BLOCK, RED, PYRAMID, and EQUIDIMENSIONAL-all parts of the sys

FIGURE 118. *Procedural representation of "a red cube which supports a pyramid." Adapted from Roger Schank and Kenneth Colby, Computer Models of Thought and Language (San Francisco: W. H. Freeman, 1973), p. 172.*



tem's underlying categorization of the world. The result can be represented in a flow chart like that in Figure 118. Note that this is a program for finding an object fitting the description. It would_ then be incorporated into a command for doing something with the object, a question asking something about it, or, if it appeared in a statement, it would become part of the program which was generated to represent the meaning for later use. Note that this bit of program could also be used as a test to see whether an object fit the description, if the first **FIND** instruction were told in advance to look only at that particular object.

At first glance, it seems that there is too much structure in this program, as we don't like to think of the meaning of a simple phrase as explicitly containing loops, conditional tests, and other programming details. The solution is to provide an internal language that contains the appropriate looping and checking as its primitives, and in which the representation of the process is as simple as the description. The program described in Figure 11S would be written in **PLANNER** looking something like what is below:

```
(GOAL (IS ?X 1 BLOCK))
(GOAL (COLOR-OF ?X1 RED))
(GOAL (EQUIDIMENSIONAL ?X1))
(GOAL (IS ?X2 PYRAMID))
(GOAL (SUPPORT ?X1 ?X2))
```

The loops of the flowchart are implicit in **PLANNER**'S backtrack control structure. The description is evaluated by proceeding down the list until some goal fails, at which time the system backs up automatically to the last point where a decision was made, trying a different possibility. A decision can be made whenever a new object name or **VARIABLE** (indicated by the prefix
") such as "?X 1" or "?X2" appears. The variables are used by the pattern matcher. If' they have already been assigned to a particular item, it checks to see whether the **GOAL** is true for that item. If not, it checks for all possible items which satisfy the **GOAL**, by choosing one, and then taking successive ones whenever backtracking occurs to that point. Thus, even the distinction between testing and choosing is implicit.[21]

One significant strategy decision in devising this program was to not translate all the way from English into **LISP**, but only partway-into **PLANNER**. Thus (since the **PLANNER** interpreter is itself written in **LISP**), a new intermediate level-**PLANNER**-was inserted between the top-level language (English) and the bottom-level language (machine language). Once a **PLANNER** program had been made from an English sentence fragment, then it could be sent off to the **PLANNER** interpreter, and the higher levels of **SHRDLU** would be freed up, to work on new tasks.

This kind of decision constantly crops up: How many levels should a system have? How much and what kind of "intelligence" should be placed on which level? These are some of the hardest problems facing **AI** today. Since we know so little about natural intelligence, it is hard for us to figure out which level of an artificially intelligent system should carry out what part of a task.

This gives you a glimpse behind the scenes of the Dialogue preceding this Chapter. Next Chapter, we shall meet new and speculative ideas for **AI**.

Contrafactus

The Crab has invited a small group of friends over to watch the Saturday afternoon football game on television. Achilles has already arrived, but the Tortoise and his friend the Sloth are still awaited.

Achilles: Could that be our friends, a-riding up on that unusual one-wheeled vehicle?

(The Sloth and Tortoise dismount and come in.)

Crab: Ah, my friends, I'm so glad you could make it. May I present my old and beloved acquaintance, Mr. Sloth-and this is Achilles. I believe you know the Tortoise.
Sloth: This is the first time I can recall making the acquaintance of a Bicyclops. Pleased to meet you, Achilles. I've heard many fine things said about the bicyclopean species.
Achilles: Likewise, I'm sure. May I ask about your elegant vehicle? Tortoise: Our tandem unicycle, you mean? Hardly elegant. It's just a way for two to get from A to B, at the same speed.
Sloth: It's built by a company that also makes teeter-teeters.
Achilles: I see, I see. What is that knob on it?
Sloth: That's the gearshift.
Achilles: Aha! And how many speeds does it have?
Tortoise: One, including reverse. Most models have fewer, but this is a special model.
Achilles: It looks like a very nice tandem unicycle. Oh, Mr. Crab, I wanted to tell you how much I enjoyed hearing your orchestra perform last night.
Crab: Thank you, Achilles. Were you there by any chance, Mr. Sloth? Sloth: No, I couldn't make it, I'm sad to say. I was participating in a mixed singles ping-ping tournament. It was quite exciting because my team was involved in a one-way tie for first place.
Achilles: Did you win anything?
Sloth: Certainly did-a two-sided Mobius strip made out of copper; it is silver-plated on one side, and gold-plated on the other. Crab: Congratulations, Mr. Sloth.
Sloth: Thank you. Well, do tell me about the concert.
Crab: It was a most enjoyable performance. We played some pieces by the Bach twins
Sloth: The famous Job and Sebastian?
Crab: One and the same. And there was one work that made me think of you, Mr. Sloth-a marvelous piano concerto for two left hands. The

next-to-last (and only) movement was a one-voice fugue. You can't imagine its intricacies. For our finale, we played Beethoven's Ninth Zenfunny. At the end, everyone in the audience rose and clapped with one hand. It was overwhelming.

Sloth: Oh, I'm sorry I missed it. But do you suppose it's been recorded: At home I have a fine hi-fi to play it on-the best two-channel monaural system money can buy.

Crab: I'm sure you can find it somewhere. Well, my friends, the game is about to begin.

Achilles: Who is playing today, Mr. Crab?

Crab: I believe it's Home Team versus Visitors. Oh, no-that was last week. I think this week it's Out-of-Towners.

Achilles: I'm rooting for Home Team. I always do.

Sloth: Oh, how conventional. I never root for Home Team. The closer a team lives to the antipodes, the more I root for it.

Achilles: Oh, so you live in the Antipodes? I've heard it's charming to live there, but I wouldn't want to visit them. They're so far away.

Sloth: And the strange thing about them is that they don't get any closer no matter which way you travel.

Tortoise: That's my kind of place.

Crab: It's game time. I think I'll turn on the TV.

(He walks over to an enormous cabinet with a screen, underneath which is an instrument panel as complicated as that of a jet airplane. He flicks a knob, and the football stadium a ears in bright vivid color on the screen.)

Announcer: Good afternoon, fans. Well, it looks like that time of year has rolled around again when Home Team and Out-of-Town face each other on the gridiron and play out their classic pigskin rivalry. It's been drizzling on and off this afternoon, and the field's a little wet, but despite the weather it promises to be a fine game, especially with that GREAT pair of eighth-backs playing for Home Team, Tedzilliger and Palindromi. And now, here's Pilipik, kicking off for Home Team. It's in the air! Flampson takes it for Out-of-Towners, and runs it back he's to the 20, the 25, the 30, and down at the 32. That was Mool in on the tackle for Home Team.

Crab: A superb runback! Did you see how he was ALMOST tackled by Quilker-but somehow broke away?

Sloth: Oh, don't be silly, Crab. Nothing of the kind happened. Quilker did NOT tackle Flampson. There's no need to confuse poor Achilles (or the rest of us) with hocus-pocus about what "almost" happened. It's a fact-with no "almost" 's, "if "'s, "and" 's, or "but" 's.

Announcer: Here's the instant replay. Just watch number 79, Quilker, come in from the side, surprising Flampson, and just about tackle him!

Sloth: "Just about"! Bah!

'Achilles: Such a graceful maneuver! What would we do without instant replays?

Announcer: It's first down and 10 for Out-of-Town. Noddle takes the ball, hands off to Orwix-it's a reverse-Orwix runs around to the right, handing off to Flampson-a double reverse, folks!-and now
Flampson hands it to Treefig, who's downed twelve yards behind scrimmage. A twelve-yard loss on a triple reverse!
Sloth: I love it! A sensational play!
Achilles: But, Mr. S, I thought you were rooting for Out-of-Town. They lost twelve yards on the play.
Moth: They did? Oh, well-who cares, as long as it was a beautiful play?
Let's see it again.

(... and so the first half of the game passes. Towards the end of the third quarter, a particularly crucial play comes up for Home Team. They are behind by eight points. It's third down and 10, and they badly need a first down.)

Announcer: The ball is hiked to Tedzilliger, who fades back, looking-for a receiver, and fakes to Quilker. There's Palindromi, playing wide right, with nobody near him. Tedzilliger spots him and fires a low pass to him. Palindromi snatches it out of the air, and- (*There is an audible groan from the crowd*.)--oh, he steps out of bounds! What a crushing blow for Home Team, folks! If Palindromi hadn't stepped out of bounds, he could've run all the way to the end zone for a touchdown!
Let's watch the subjunctive instant replay.

(And on the screen the same lineup appears as before.)

The ball is hiked to Tedzilliger, who fades back, looking for a receiver, and fakes to Quilker. There's Palindromi, playing wide right, with nobody near him. Tedzilliger spots him, and fires a low pass to him. Palindromi snatches it out of the air, and- (*There is an audible gasp from the crowd.*)-he almost steps out of bounds! But he's still in bounds, and it's clear all the way to the end zone! Palindromi streaks in, for a touchdown for Home Team! (The stadium breaks into a giant roar of approval.) Well, folks, that's what would've happened if Palindromi hadn't stepped out of bounds.

Achilles: Wait a minute ... WAS there a touchdown, or WASN'T there?
Crab: Oh, no. That was just the subjunctive instant replay. They simply followed a hypothetical a little way out, you know.
Sloth: That is the most ridiculous thing I ever heard of! Next thing you know, they'll be inventing concrete earmuffs.
Tortoise: Subjunctive instant replays are a little unusual, aren't they?
Crab: Not particularly, if you have a Subjunc-TV.
Achilles: Is that one grade below a junk TV?
Crab: Not at all! It's a new kind of TV, which can go into the subjunctive mode. They're particularly good for football games and such. I just got mine.
Achilles: Why does it have so many knobs and fancy dials?

Crab: So that you can tune it to the proper channel. There are many channels broadcasting in the subjunctive mode, and you want to be able to select from them easily.

Achilles: Could you show us what you mean? I'm afraid I don't quite understand what all this talk of "broadcasting in the subjunctive mode" is about.

Crab: Oh, it's quite simple, really. You can figure it out yourself. I'm going into the kitchen to fix some French fries, which I know are Mr. Sloth's weakness.

Sloth: Mmmmm! Go to it, Crab! French fries are my favorite food. Crab: What about the rest of you?

Tortoise: I could devour a few.

Achilles: Likewise. But wait-before you go into the kitchen, is there some trick to using your Subjunc-TV?

Crab: Not particularly. Just continue watching the game. and whenever there's a near miss of some sort, or whenever you wish things had gone differently in some way, just fiddle with the dials, and see what happens. You can't do it any harm, though you may pick up some exotic channels. (*And he disappears into the kitchen*.)

Achilles: I wonder what he means by that. Oh well, let's get back to this game. I was quite wrapped up in it.

Announcer: It's fourth down for Out-of-Town, with Home Team receiving. Out-of-Town is in punt formation, with Tedzilliger playing deep. Orwix is back to kick-and he gets a long high one away. It's coming down near Tedzilliger

Achilles: Grab it, Tedzilliger! Give those Out-of-Towners a run for their money!

Announcer: -and lands in a puddle-KERSPLOSH! It takes a weird bounce! Now Sprunk is madly scrambling for the ball! It looks like it just barely grazed Tedzilliger on the bounce, and then slipped away from himit's ruled a fumble. The referee is signaling that the formidable Sprunk has recovered for Out-of-Town on the Home Team 7! It's a bad break for Home Team. Oh, well, that's the way the cookie crumbles.

Achilles: Oh, no! If only it hadn't been raining ... (*Wrings his hands in despair*.)

Sloth: ANOTHER of those confounded hypotheticals! Why are the rest of you always running off into your absurd worlds of fantasy? If I were you, I would stay firmly grounded in reality. "No subjunctive nonsense" is my motto. And I wouldn't abandon it even if someone offered me a hundred-nay, a hundred and twelve-French fries.

Achilles: Say, that gives me an idea. Maybe by suitably fiddling with these knobs, I can conjure up a subjunctive instant replay in which it isn't raining, there's no puddle, no weird bounce, and Tedzilliger doesn't

fumble. I wonder ... (*Walks up to the Subjunc-TV and stares at it.)* But I haven't any idea what these different knobs do. (*Spins a few at random*.)

Announcer: It's fourth down for Out-of-Town, with Home Team receiv-

ing. Out-of-Town is in punt formation, with Tedzilliger playing deep. Orwix is back to kick-and he gets a long high one away. It's coming down near Tedzilliger

Achilles: Grab it, Tedzilliger! Give those Out-of-Towners a run for their money!

Announcer: -and lands in a puddle-KERSPLOSH! Oh-it bounces right into his arms! Now Sprunk is madly scrambling after him, but he's got - good blocking, and he steers his way clear of the formidable Sprunk, and now he's got an open field ahead of him. Look at that, folks! He's to the 50, the 40, the 30, the 20, the 10-touchdown, Home Team! (*Huge cheers from the Home Team side*.) Well, fans, that's how it would have gone, if footballs were spheres instead of oblate spheroids! But in reality, Home Team loses the ball, and Out-of-Towners take over on the Home Team 7-yard line. Oh, well, that's the way the ball bounces.

Achilles: What do you think of THAT, Mr. Sloth?

(And Achilles gives a smirk in the direction of the Sloth, but the latter is completely oblivious to its devastating effect, as he is busy watching ,the Crab arrive with, a large platter with a hundred and twelve-nay, a hundred-large and delicious French fries, and napkins for all.)

Crab: So how do you three find my Subjunc-TV?

Sloth: Most disappointing, Crab, to be quite frank. It seems to be badly out of order. It makes pointless excursions into nonsense at least half the time. If it belonged to me, I would give it away immediately to someone like you, Crab. But of course it doesn't belong to me.

Achilles: It's quite a strange device. I tried to rerun a play to see how it would have gone under different weather conditions, but the thing seems to have a will of its own! Instead of changing the weather, it changed the football shape to ROUND instead of FOOTBALL-SHAPED! Now tell me-how can a football not be shaped like a football? That's a contradiction in terms. How preposterous!

Crab: Such tame games! I thought you'd surely find more interesting subjunctives. How would you like to see how the last play would have looked if the game had been baseball instead of football?

Tortoise: Oh! An outstanding idea!

(The Crab twiddles two knobs, and steps back.)

Announcer: There are four away, and---

Achilles: FOUR away!?

Announcer: That's right, fans-four away. When you turn football into baseball, SOMETHING'S got to give! Now as I was about to say, there are four away, with Out-of-Town in the field, and Home Team up. Tedzilliger is at bat. Out-of-Town is in bunt formation. Orwix raises his arm to pitch-and he gets a long high ball away. It's heading straight for Tedzilliger

Achilles: Smash it, Tedzilliger! Give those Out-of-Towners a home run for their money!

Announcer: -but it seems to be a spitball, as it takes a strange curve. Now Sprunk is madly scrambling for the ball! It looks like it just barely grazed Tedzilliger's bat, then bounced off it-it's ruled a fly ball. The umpire is signaling that the formidable Sprunk has caught it for Out-of-Town, to end the seventh inning. It's a bad break for Home Team. That's how the last play would have looked, football fans, if this had been a game of baseball.
Sloth: Bah! You might as well transport this game to the Moon.
Crab: No sooner said than done! Just a twiddle here, a twiddle there ...

(On the screen there appears a desolate crater-pitted field, with two teams in space suits facing each other, immobile. All at once, the two teams fly into motion, and the players are making great bounds into the air, sometimes over the heads of other players. The ball is thrown into the air, and sails so high that it almost disappears, and then slowly comes floating down into the arms of one space-suited player, roughly a quarter-mile from where it was released.)

Announcer: And there, friends, you have the subjunctive instant replay as it would have happened on the Moon. We'll be right back after this important commercial message from the friendly folks who brew Glumpf Beer-my favorite kind of beer!
Sloth: If I weren't so lazy, I would take that broken TV back to the dealer myself! But alas, it's my fate to be a lazy Sloth ... (*Helps himself to a large gob of French fries*.)
Tortoise: That's a marvelous invention, Mr. Crab. May I suggest a hypothetical?
Crab: Of course!
Tortoise: What would that last play have looked like if space were four-dimensional?
Crab: Oh, that's a complicated one, Mr. T, but I believe I can code it into the dials. Just a moment.

(He steps up, and, for the first time, appears to be using the full power of the control panel of his Subjunc-TV, turning almost every knob two or three times, and carefully checking various meters. Then he steps back with a satisfied expression on his face.)

I think this should do it.

Announcer: And now let's watch the subjunctive instant replay.

(A confusing array of twisted pipes appears on the screen. It grows larger, then smaller, and for a moment seems to do something akin to rotation. Then it turns into a strange mushroom-shaped object, and back to a bunch of pipes. As it metamorphoses from this into other bizarre shapes, the announcer gives his commentary.)

Tedzilliger's fading back to pass. He spots Palindromi ten yards outfield, and passes it to the right and outwards-it looks good! Palindromi's at the 35-yard plane, the 40, and he's tackled on his own

43-yard plane. And there you nave it, 3-L tans, as it would have looked if football were played in four spatial dimensions.

Achilles: What is it you are doing, Mr. Crab, when you twirl these various dials on the control panel?

Crab: I am selecting the proper subjunctive channel. You see, there are all sorts of subjunctive channels broadcasting simultaneously, and I want to tune in precisely that one which represents the kind of hypothetical which has been suggested.

Achilles: Can you do this on any TV?

Crab: No, most TV's can't receive subjunctive channels. They require a special kind of circuit which is quite difficult to make.

Sloth: How do you know which channel is broadcasting what? Do you look it up in the newspaper?

Crab: I don't need to know the channel's call letters. Instead, I tune it in by coding, in these dials, the hypothetical situation which I want to be represented. Technically, this is called "addressing a channel by its counterfactual parameters". There are always a large number of channels broadcasting every conceivable world. All the channels which carry worlds that are "near" to each other have call letters that are near to each other, too.

Tortoise: Why did you not have to turn the dials at all, the first time we saw a subjunctive instant replay?

Crab: Oh, that was because I was tuned in to a channel which is very near to the Reality Channel, but ever so slightly off. So every once in a while, it deviates from reality. It's nearly impossible to tune EXACTLY into the Reality Channel. But that's all right, because it's so dull. All their instant replays are straight! Can you imagine? What a bore!

Sloth: I find the whole idea of Subjunc-TV's one giant bore. But perhaps I could change my mind, if I had some evidence that your machine here could handle an INTERESTING counterfactual. For example, how would that last play have looked if addition were not commutative?

Crab: Oh me, oh my! That change is a little too radical, I'm afraid, for this model. I unfortunately don't have a Superjunc-TV, which is the top of the line. Superjunc-TV's can handle ANYTHING you throw at them.

Sloth: Bah!

Crab: But look-I can do ALMOST as well. Wouldn't you like to see how the last play would have happened if 13 were not a prime number? Sloth: No thanks! THAT doesn't make any sense! Anyway, if I were the last play, I'd be getting pretty tired of being trotted out time and again in new garb for the likes of you fuzzy-headed concept-slippers. Let's get on with the game!

Achilles: Where did you get this Subjunc-TV, Mr. Crab?

Crab: Believe it or not, Mr. Sloth and I went to a country fair the other evening, and it was offered as the first prize in a lottery. Normally I don't indulge in such frivolity, but some crazy impulse grabbed me, and I bought one ticket.

Achilles: What about you Mr. Sloth?
Sloth: I admit, I bought one, just to humor old Crab.
Crab: And when the winning number was announced, I found, to my amazement, that I'd won the lottery!
Achilles: Fantastic! I've never known anyone who won anything in a lottery before!
Crab: I was flabbergasted at my good fortune.
Sloth: Don't you have something else to tell us about that lottery, Crab?
Crab: Oh, nothing much. It's just that my ticket number was 129. Now when they announced the winning number, it was 128 just one off. Sloth: So you see, he actually didn't win it at all. Achilles: He ALMOST won, though ...
Crab: I prefer to say that I won it, you see. For I came so terribly close . . If my number had been only one smaller, I would have won. Sloth: But unfortunately, Crab, a miss is as good as a mile.
Tortoise: Or as bad. What about you, Mr. Sloth? What was your number:
Sloth: Mine was 256-the next power of 2 above 128. Surely, that counts as a hit, if anything does! I can't understand why, however, those fair officials-those UNfair officials-were so thickheaded about it. They refused to award me my fully deserved prize. Some other joker claimed HE deserved it, because his number was 128. 1 think my number was far closer than His, but you can't fight City Hall.
Achilles: I'm all confused. If you didn't win the Subjunc-TV after all, Mr. Crab, then how can we have been sitting here all afternoon watching it? It seems as if we ourselves have been living in some sort of hypothetical world that would have been, had circumstances just been ever so slightly different ...
Announcer: And that, folks, was how the afternoon at Mr. Crab's would have been spent, had he won the Subjunc-TV. But since he didn't, the four friends simply spent a pleasant afternoon watching Home Team get creamed, 128-0. Or was it 256-0? Oh well, it hardly matters, in five-dimensional Plutonian steam hockey.

CHAPTER XIX

Artificial Intelligence: Prospects

"Almost" Situations and Subjunctives

AFTER READING *Contrafactus*, a friend said to me, "My uncle was almost President of the U.S.!" "Really?" I said. "Sure," he replied, "he was skipper of the PT 108." (John F. Kennedy was skipper of the PT 109.)

That is what *Contrafactus* is all about. In everyday thought, we are constantly manufacturing mental variants on situations we face, ideas we have, or events that happen, and we let some features stay exactly the same while others "slip". What features do we let slip? What ones do we not even consider letting slip? What events are perceived on some deep intuitive level as being close relatives of ones which really happened? What do we think "almost" happened or "could have" happened, even though it unambiguously did not? What alternative versions of events pop without any conscious thought into our minds when we hear a story? Why do some counterfactuals strike us as "less counterfactual" than other counterfactuals? After all, it is obvious that anything that didn't happen didn't happen. There aren't degrees of "didn't-happen-ness". And the same goes for "almost" situations. There are times when one plaintively says, "It almost happened", and other times when one says the same thing, full of relief. But the "almost" lies in the mind, not in the external facts.

Driving down a country road, you run into a swarm of bees. You don't just duly take note of it; the whole situation is immediately placed in perspective by a swarm of "replays" that crowd into your mind. Typically, you think, "Sure am lucky my window wasn't open!"-or worse, the reverse: "Too bad my window wasn't closed!" "Lucky I wasn't on my bike!" "Too bad I didn't come along five seconds earlier." Strange but-possible replays: "If that had been a deer, I could have been killed!" "I bet those bees would have rather had a collision with a rosebush." Even stranger replays: "Too bad those bees weren't dollar bills!" "Lucky those bees weren't made of cement!" "Too bad it wasn't just one bee instead of a swarm." "Lucky I wasn't the swarm instead of being me." What slips naturally and what doesn't-and why?

In a recent issue of *The New Yorker* magazine, the following excerpt from the "Philadelphia Welcomat" was reprinted:'

> If Leonardo da Vinci had been born a female the ceiling of the Sistine Chapel might never have been painted.[1]